用户指南

C2000™ 微控制器引导加载入门指南

Matt Kukucka

摘要

嵌入式处理器在很多情况下需要进行编程，此时 JTAG 调试探针可能无法可靠地为目标器件烧录程序。在这种情况下，工程师需要借助 USB（通用串行总线）或灵活数据速率控制器局域网（CAN-FD）等外设实现编程。C2000™ 器件通过 Boot ROM 中的多种引导加载实用程序，可将固件加载到片上 RAM，从而满足这一需求。这些实用程序很有用，但初次接触时可能不易理解，实际调试也可能较为困难。本文介绍基本的引导加载配置，并说明如何利用常见引导模式将应用程序代码加载到片上闪存。

目录

1 引言 4
2 配置引导模式 6
2.1 独立引导 6
2.1.1 引导模式选择引脚（BMSP） 7
2.1.2 引导定义表（BOOTDEF） 8
2.1.3 Boot ROM OTP 配置寄存器 10
2.1.4 CPU2 引导流程 10
2.2 仿真引导 12
3 闪存编程 13
3.1 Flash API 14
3.2 闪存内核 14
4 将代码引导加载到闪存 16
4.1 C2000 十六进制实用程序 16
4.2 常见引导模式 17
4.2.1 引导至闪存 17
4.2.2 SCI 引导 20
4.2.3 CAN 引导 27
4.2.4 CAN-FD 引导 35
4.2.5 USB 引导 42
5 常见问题 48
5.1 通过软件实现选择 BMSP GPIO 48
5.2 从闪存而非 RAM 运行闪存内核 48
5.3 调试 Boot ROM 时未定义符号 51
5.4 使用片上闪存工具向 OTP 写入数值 53
5.5 使用 Flash API 插件向 OTP 写入数值 55
6 总结 56
7 参考资料 57

版权所有 © 2025 Texas Instruments Incorporated

插图清单

图 1-1. 通用引导加载程序设计流程 ····· 5
图 3-1. CCS v12 中片上闪存工具的位置 ····· 13
图 3-2. CCS v20 中片上闪存工具的位置 ····· 13
图 4-1. 添加用于调用十六进制实用程序的构建后步骤 ····· 16
图 4-2. 在 CCS 中打开目标配置菜单 ····· 18
图 4-3. 在 CCS 中启动目标配置 ····· 18
图 4-4. 在 CCS 中连接目标内核 ····· 18
图 4-5. 在 CCS 中打开存储器浏览器 ····· 19
图 4-6. 模拟零引脚闪存引导（0x0009 0000） ····· 19
图 4-7. 查找转换后的 SCI 内核输出文件 ····· 21
图 4-8. F2800157 LaunchPad UART 路由原理图 ····· 22
图 4-9. 将 SCI TX/RX（GPIO28/GPIO29）连接到 BoosterPack（BP） 22
图 4-10. 将 XDS TX/RX 跳线接到 SCI TX/RX GPIO ····· 23
图 4-11. 在 CCS 中打开目标配置菜单 ····· 23
图 4-12. 在 CCS 中启动目标配置 ····· 24
图 4-13. 在 CCS 中连接目标内核 ····· 24
图 4-14. 在 CCS 中打开存储器浏览器 ····· 24
图 4-15. 零引脚 SCI 引导：SCIRXDA 接 GPIO28，SCITXDA 接 GPIO29 25
图 4-16. 在设备管理器中查找 XDS COM 端口 ····· 25
图 4-17. 使用串行闪存编程器下载应用程序 ····· 26
图 4-18. 应用程序写入闪存后运行 ····· 26
图 4-19. F280039C LaunchPad CAN 收发器和连接器原理图 ····· 28
图 4-20. 为正确的 LaunchPad CAN GPIO 分配添加预定义符号 ····· 29
图 4-21. 包含本地 CAN 时序头文件 ····· 29
图 4-22. 确认 CAN 时序设置 ····· 29
图 4-23. 生成 DCAN 内核 txt 输出 ····· 30
图 4-24. 设置 CAN 路由开关 ····· 31
图 4-25. 在 CCS 中打开目标配置菜单 ····· 31
图 4-26. 在 CCS 中启动目标配置 ····· 32
图 4-27. 在 CCS 中连接目标内核 ····· 32
图 4-28. 在 CCS 中打开存储器浏览器 ····· 32
图 4-29. 零引脚 CAN 引导：CANRXA=GPIO5，CANTXA=GPIO4 ····· 33
图 4-30. 已加载 DCAN 闪存编程器内核 ····· 34
图 4-31. 已加载 DCAN 闪存编程器应用程序 ····· 34
图 4-32. F280039C LaunchPad CAN 收发器和连接器原理图 ····· 36
图 4-33. 为正确的 LaunchPad CAN GPIO 分配添加预定义符号 ····· 37
图 4-34. 生成 CAN 内核 txt 输出 ····· 37
图 4-35. 设置 CAN 路由开关 ····· 38
图 4-36. 在 CCS 中打开目标配置菜单 ····· 39
图 4-37. 在 CCS 中启动目标配置 ····· 39
图 4-38. 在 CCS 中连接目标内核 ····· 39
图 4-39. 在 CCS 中打开存储器浏览器 ····· 40
图 4-40. 零引脚 MCAN 引导：CANRXA=GPIO5，CANTXA=GPIO4 ····· 40
图 4-41. 已加载 MCAN 闪存编程器内核 ····· 42
图 4-42. 已加载 MCAN 闪存编程器应用程序 ····· 42
图 4-43. 生成正确的 USB 内核 Bin 输出 ····· 43
图 4-44. 通过 CCS 和板载 XDS 调试 F2837xD controlCARD ····· 44
图 4-45. 在 CCS 中打开目标配置菜单 ····· 44
图 4-46. 在 CCS 中启动目标配置 ····· 45
图 4-47. 在 CCS 中连接目标内核 ····· 45
图 4-48. 在 CCS 中打开存储器浏览器 ····· 45
图 4-49. 配置 F2837xD controlCARD 以模拟 USB 引导 ····· 46
图 4-50. 在 C2000Ware 中查找 USB Windows 驱动程序 ····· 46
图 4-51. 设备管理器中显示的 F28x7x USB Boot Loader ····· 47
图 5-1. 在 CCS 项目中添加文件 ····· 49
图 5-2. 在编译器构建中排除 RAM 链接命令文件 ····· 49
图 5-3. 为项目创建新的构建配置 ····· 50
图 5-4. 创建 CPU1_FLASH 构建配置 ····· 50
图 5-5. 添加预定义符号 _FLASH 以初始化闪存函数 ····· 51
图 5-6. 在 CCS 中打开目标配置菜单 ····· 52
图 5-7. 在 CCS 中启动目标配置 ····· 52
图 5-8. 在 CCS 中连接目标内核 ····· 52
图 5-9. 未加载 Boot ROM 符号时的 CCS 视图 ····· 52
图 5-10. 在 CCS 中查找“加载符号”命令 ····· 53
图 5-11. 查找 Boot ROM 源文件 ····· 53
图 5-12. 示例 1：已配置的闪存插件引导 ····· 54
图 5-13. 示例 2：已配置的闪存插件引导 ····· 55

版权所有 © 2025 Texas Instruments Incorporated

表格清单

表 2-1. F280015x 器件默认引导模式 6
表 2-2. 器件默认引导模式选择引脚 6
表 2-3. 各器件系列的引导配置类型 7
表 2-4. BOOTPIN-CONFIG 位字段 8
表 2-5. BOOTDEF 位字段 9
表 2-6. F280015x 器件引导模式 9
表 2-7. F280015x Boot ROM 寄存器 10
表 2-8. CPU1TOCPU2IPCBOOTMODE 寄存器详细信息 11
表 2-9. 仿真引导寄存器位置 12
表 5-1. 零引脚引导配置结果 54
表 5-2. 双引导模式选择引脚配置 55

商标

C2000™、LaunchPad™、Code Composer Studio™ 和 BoosterPack™ 均为德州仪器的商标。
Microsoft Visual Studio® 是 Microsoft Corporation 的注册商标。所有商标均归其各自所有者所有。

版权所有 © 2025 Texas Instruments Incorporated

1 引言

> 注意：本文档仅适用于基于 C28x 的微控制器。有关 C29x 引导加载的详细信息，请参阅器件专用技术参考手册（TRM）。

随着应用复杂度不断提高，在现场维护器件的安全性和信息安全完整性时，故障修复、功能扩展和嵌入式固件修改能力变得越来越重要。C2000 器件通过在引导

ROM（出厂编程的只读存储器）中提供简单的固件加载实用程序，实现固件更新。

ROM 加载器通常称为引导加载程序，位于目标器件的 Boot ROM 中，可通过软件从外部主机加载应用程序代码。引导加载程序可替代 JTAG 调试探针；后者需要昂贵且专用的硬件，并须直接连接目标器件。

虽然具体可用选项因器件而异，但用户可选择多种外设引导加载程序，将固件加载到目标器件，例如：

1. 串行通信接口（SCI） 2. 串行外设接口（SPI） 3. 集成电路总线（I2C） 4. 控制器局域网（CAN） 5. 灵活数据速率控制器局域网（CAN-FD） 6. 通用串行总线（USB） 7. 并行 GPIO

每种 C2000 器件都提供一组可选的默认引导模式。不过，如果用户需要使用默认引导表中未提供的引导模式，或需要灵活配置 GPIO 分配，
就必须配置一次性可编程（OTP）存储器。若默认引导表未提供所需模式，可通过 OTP 寄存器选择其他引导模式。

如果用户选择使用外设引导加载程序将新代码加载到器件，则必须事先按特定格式生成应用映像，具体说明见第 4.1 节。应用准备就绪后，Boot ROM 与主机之间即可通过选定的外设引导加载程序传输数据。随后，引导加载程序会将应用程序代码加载到片上 RAM 并执行。

每种 C2000 器件的 ROM 中都包含外设引导加载程序，使用简单，但仅限于
将代码加载到 RAM。闪存内核可作为中间环节，在 ROM 与闪存之间搭建桥梁，使代码可从 RAM 写入闪存，具体说明见第 3.2 节。

不过，刚开始接触 C2000 引导加载的用户容易遇到问题；若引导流程中任何一步出错，排查起来可能很困难（总体设计流程见图 1-1）。本文将说明引导加载程序设计的四个阶段：

1. 选择并配置适当的引导模式 2. 准备应用程序，使其可由 Boot ROM 中的外设引导加载程序加载 3. 使用引导加载程序将应用加载到 RAM 4. 使用闪存内核对闪存编程

版权所有 © 2025 Texas Instruments Incorporated

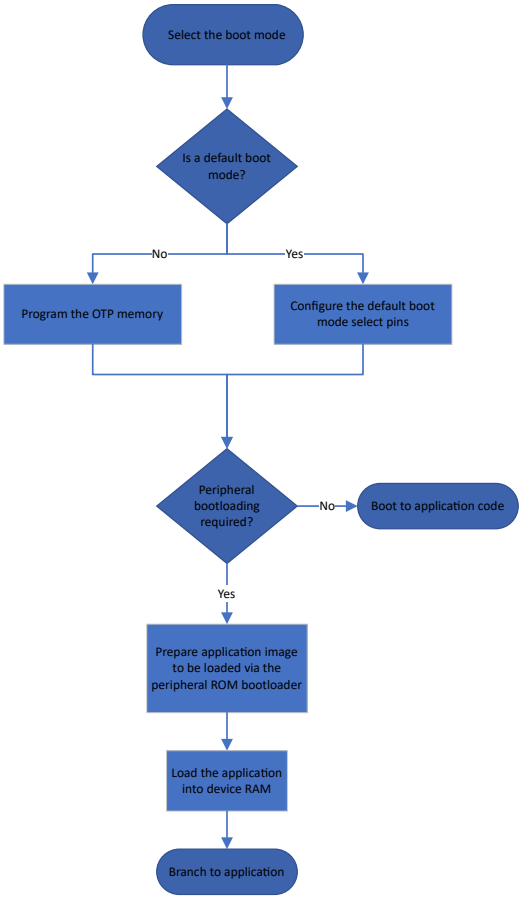

图 1-1. 通用引导加载程序设计流程

版权所有 © 2025 Texas Instruments Incorporated

2 配置引导模式

在 ROM 引导序列结束时，器件会根据是否连接 JTAG 调试器，决定进入仿真引导还是独立引导。除 F2837xD/F2837xS/F2807x 器件通过轮询“TRSTn”引脚外，其他器件通过读取 JTAG 状态机中的“DCON”位完成判断。仿真引导会模拟独立引导流程：它从 RAM 中读取寄存器，这些寄存器的结构和配置与一次性可编程（OTP）存储器中的引导寄存器相同。因此，仿真引导允许用户根据需要多次写入引导寄存器。

注意：TI 强烈建议先使用仿真模式调试并验证器件引导配置是否正确，再尝试对 OTP 编程，因为 OTP 只能编程一次。

本章详细介绍独立引导（第 2.1 节）和仿真引导（第 2.2 节）的配置与用法。请注意，仿真引导流程将在第 4.2 节中演示。

2.1 独立引导

注

本文以 F280015x 器件系列为例，但内容也适用于任何使用 BOOTPIN-CONFIG/BOOTDEF 寄存器的器件（见表 2-3）。器件专属信息请参阅其技术参考手册（TRM）的 *Boot ROM* 章节。

每次 CPU 复位时，器件都会根据复位类型和引导配置，在 Boot ROM 中执行预定义的引导序列。器件成功初始化后（假设未连接调试器），
器件会轮询引导模式选择引脚（BMSP），以确定要调用的引导模式。BMSP 可映射到外部 GPIO 引脚；引脚分配由器件默认配置（表 2-1）或用户在 OTP 存储器中的自定义设置决定（详见第 2.1.1 节）。此过程称为独立引导，它比直接连接调试器更灵活，可通过器件的外设模块执行固件更新。

所有较新的 C2000 器件都预先配置了引导模式，可通过各器件系列的默认 BMSP 进行选择（见表 2-2）。使用出厂默认引导模式时，器件可直接从闪存启动，也可通过预先确定的外设通信模块将新的应用程序代码加载到 RAM，无需编写任何引导寄存器。但如果用户需要使用默认引导模式未涵盖的外设，或需要进一步自定义引导选项，该怎么办?

表 2-1. F280015x 器件默认引导模式

Boot Mode	GPIO24 (Default boot mode select pin 1)	GPIO32 (Default boot mode select pin 0)
Parallel IO	0	0
SCI / Wait Boot	0	1
CAN	1	0
Flash	1	1

表 2-2. 器件默认引导模式选择引脚

	GPIO 24 and GPIO 32	GPIO 72 and GPIO 84
Device Families	F28002x, F28003x, F28004x, F280013x, F280015x, F28P55x	F2837xD, F2837xS, F2807x, F2838x, F28P65x

版权所有 © 2025 Texas Instruments Incorporated

2.1.1 Boot Mode Select Pins (BMSP)

器件会解码引导模式选择引脚（BMSP），并以其值作为索引查找引导定义表，从而确定要执行的引导模式。除每种器件均提供的默认引导配置外，用户还可通过编程 OTP 存储器，选择其他 BMSP 和引导模式。ROM 的存储器内容在制造时确定；OTP 则可在生产后编程一次，因此在需要可靠、可重复读取数据的应用中提供了更大的灵活性。

在独立引导加载场景中，遇到以下情况时需要对 OTP 编程：

1. 默认引导选项不支持所需的引导模式 2. 外设或 BMSP 需要使用不同的 GPIO 3. 应用程序需要使用不同的入口点 4. 需要灵活使用多个引导选项

根据器件系列（见表 2-3），可通过写入用户可配置的双代码安全模块（DCSM）OTP 中相应的 BOOTPIN-CONFIG 或 BOOTCTRL 存储器位置，修改 BMSP。[7]

注

DCSM 中有两个独立安全区，可分配受保护资源：区域 1（Z1）和区域 2（Z2）。安全模块会限制 CPU 对片上安全存储器和资源的访问，但不会中断或暂停 CPU 执行。读取安全存储器位置时，返回值为 0，CPU 随即继续执行下一条指令。这样可阻止通过 JTAG 端口读写安全存储器。请注意，不安全资源仍可由 JTAG 调试器访问。

BOOTPIN-CONFIG 寄存器是一个 32-bit 宽的存储位置，由四个 8-bit 分区组成。其中三个分区用于指定 BMSP，最后一个分区是用于标记 OTP 配置有效性的密钥。根据所配置的区域，该寄存器位于 DCSM OTP 中，名称为 Z1-OTP-BOOTPIN-CONFIG 或 Z2-OTP-BOOTPIN-CONFIG。

注

写入 Z2 的配置优先级高于 Z1。因此，如果需要更改 OTP 配置，TI 建议先使用 Z1 位置，再使用 Z2。

可使用 CCS 中的 On-Chip Flash 工具或 Flash API 对 DCSM OTP 中的 BOOTPIN-CONFIG 寄存器编程（具体步骤见第 5.4 节或第 5.5 节），也可通过 SysConfig 中的 DCSM 工具以图形方式配置 [8]。

表 2-3. 各器件系列的引导配置类型

BOOTPIN-CONFIG and BOOTDEF Devices	BOOTCTRL Devices
F28002x, F28003x, F28004x, F280013x, F280015x, F2838x, F28P55x, F28P65x	F2807x, F2837xD, F2837xS

引导期间，BMSP 几乎可设置为任意 GPIO（例外情况请参阅器件专用 TRM 的 *Configuring Boot Mode Pins* 章节）；其中 GPIO0 对应 0x0，GPIO1 对应 0x01，依此类推。多数情况下，必须通过外部 GPIO 引脚手动拉高或拉低 BMSP，但也可以按照第 5.1 节所述的方法，通过软件控制固件更新。

只要不违反引导模式引脚的保持时间要求，BMSP 也可在 Boot ROM 和后续运行的应用程序中复用（参见数据表中的 *Reset - XRSn - Timing Requirements*）[29]。

版权所有 © 2025 Texas Instruments Incorporated

注

不过，如果使用 LaunchPad™ 或 ControlCARD，默认 BMSP 会由外部引导开关手动上拉或下拉，因此之后无法在应用程序中安全地复用。

使用的 BMSP 数量会以指数方式扩大或限制引导表中可选择的引导模式数量。使用三个 BMSP 时，最多可选择 8 种引导选项；减少到两个 BMSP 后，只能选择四种。若不使用 BMSP，则会自动选择唯一的引导选项，无需外部控制 GPIO，也可释放其他引脚

供引导引脚以外的用途使用。

可通过向更改 GPIO 编号时所用的同一 BOOTPIN-CONFIG 存储器位置写入 *0xFF*，禁用任一 BMSP。解码引导模式时，BMSP0 是引导表索引值的最低有效位，BMSP2 是最高有效位。TI 建议禁用多个 BMSP 时，优先禁用 BMSP2。

在独立引导模式下，如果 Z1 或 Z2 OTP 加载的寄存器未写入正确的 BOOTPIN-CONFIG_KEY（0x5A）以标记配置有效，系统就会使用默认 BMSP 对默认引导表进行索引。

表 2-4. BOOTPIN-CONFIG 位字段

位段	名称	说明
31:24	密钥	将 0x5A 写入这 8 位，以告知 Boot ROM 这些寄存器位有效。
23:16	引导模式选择引脚 2（BMSP2）	参见 BMSP0 的说明。
15:8	引导模式选择引脚 1（BMSP1）	参见 BMSP0 的说明。
7:0	引导模式选择引脚 0（BMSP0）	设置为引导时使用的 GPIO 引脚（最高 255）。 0x0 = GPIO0，0x01 = GPIO1，依此类推。 写入 0xFF 可禁用此 BMSP，此引脚不再用于选择引导模式。

注

仅使用 BMSP2（禁用 BMSP1 和 BMSP0）时，可选择的引导表索引只有 0 和 4。仅使用 BMSP0 时，可选择的引导表索引为 0 和 1。配置引导表中的引导选项时，务必注意这一点：无论其他 BMSP 处于何种状态，每个 BMSP 仍具有固定的位置权重。

注

使用 BOOTCTRL 寄存器（见表 2-3）的器件，其引导流程和配置方式与使用 BOOTPIN-CONFIG 寄存器的器件系列不同。

写入有效密钥后，BOOTCTRL 寄存器允许将不同 GPIO 用作两个引导模式选择引脚。对于使用 BOOTCTRL 的器件，可自定义的 BMSP 总数固定为两个；而使用 BOOTPIN-CONFIG 的器件最多可配置三个 BMSP。不过，在所有器件上，同一 GPIO 均可同时分配给两个 BMSP，从而支持单引脚用法。

有关 BOOTCTRL 寄存器的更多详细信息，请参阅器件专用 TRM。

2.1.2 Boot Definition Table (BOOTDEF)

在 OTP 中配置 BMSP 时，还必须通过向引导定义表寄存器（BOOTDEF）写入引导选项条目来定义自定义引导模式表。用户定义的 BOOTDEF 表取代默认引导模式选择表，并根据 OTP 中自定义的 BMSP 编制索引。例如，默认配置将并行引导固定为引导选项 0；用户现在可以将第一个引导选项设置为任意可用的引导模式，其他选项依此类推。

版权所有 © 2025 Texas Instruments Incorporated

通过配置一个 64-bit 寄存器（见表 2-5）来设置 BOOTDEF 表。该寄存器位于 DCSM OTP 中，拆分为两个 32-bit 存储位置，分别称为 Z1-OTP-BOOTDEF-LOW 和 Z1-OTP-BOOTDEF-HIGH（若配置区域为 Z2，则名称分别为 Z2-OTP-BOOTDEF-LOW 和 Z2-OTP-BOOTDEF-HIGH）。随后，这些寄存器会被划分为多个 8-bit 条目，用于定义各个引导选项。

BOOTDEF 表中可自定义的引导模式范围取决于使用的 BMSP 数量。请注意，不使用 BMSP 时，表中只有一个条目；使用一个 BMSP 时，最多有两个条目；使用两个 BMSP 时，最多可有四个表条目；使用三个 BMSP 时，最多可有八个表条目。

配置 BOOTDEF 表的步骤：

1. 在数据表或技术参考手册的 *GPIO Assignments* 部分选择引导选项 2. 将该引导选项对应的 BOOTDEF 值写入预定的 BOOTDEF-LOW 或 BOOTDEF-HIGH OTP 存储位置。

可使用 CCS 中的 On-Chip Flash 工具或 Flash API 对 DCSM OTP 中的 BOOTDEF 寄存器编程（具体步骤分别见第 5.4 节和第 5.5 节），也可通过 SysConfig 中的 DCSM 工具以图形方式配置 [8]。

写入有效的 BOOTDEF 值后，复位时可使用 BOOTPIN-CONFIG 寄存器中配置的 BMSP 对引导定义表进行索引，以选择 Boot ROM 中要执行的引导选项。

表 2-5. BOOTDEF 位字段

BOOTDEF 名称	字节位置	名称	说明
BOOT_DEF0	7:0	[3:0] BOOT_DEF0 模式	设置引导模式编号（参见表 2-6）。如果使用不支持的引导模式，器件将在调试器已连接时进入等待引导模式，或在独立运行时引导至闪存。
		[7:4] BOOT_DEF0 选项	设置备用/附加引导选项。可更改特定外设引导所用 GPIO，或指定其他闪存入口点。另请参阅 GPIO Assignments 了解可用的 BOOTDEF 值。
BOOT_DEF1	15:8	BOOT_DEF1 模式/选项	参见 BOOT_DEF0 的说明。
BOOT_DEF2	23:16	BOOT_DEF2 模式/选项	
BOOT_DEF3	31:24	BOOT_DEF3 模式/选项	
BOOT_DEF4	39:32	BOOT_DEF4 模式/选项	
BOOT_DEF5	47:40	BOOT_DEF5 模式/选项	
BOOT_DEF6	55:48	BOOT_DEF6 模式/选项	
BOOT_DEF7	63:56	BOOT_DEF7 模式/选项	

表 2-6. F280015x 器件引导模式

引导编号	引导模式
0	并行 I/O
1	SCI/等待引导
2	CAN
3	闪存
4	等待引导
5	RAM
6	SPI
7	I2C
8	CAN-FD
10	安全闪存

版权所有 © 2025 Texas Instruments Incorporated

注

根据器件系列不同，引导选择表的可配置能力也存在以下例外情况：

1. F2833x 器件的引导表不可自定义，仅限使用出厂默认配置。 2. 对于 F2802x、F2803x、F2806x、F2837xD、F2837xS 和 F2807x，默认引导表仅部分可自定义：可将第 4 个条目（GET 模式）设置为另一种引导模式。

这与使用 BOOTDEF 寄存器的器件不同（见表 2-3）；后者最多可选择八种引导模式。有关 BOOTCTRL 的更多详细信息，请参阅器件专用 TRM。

2.1.3 Boot ROM OTP Configuration Registers

Boot ROM 代码执行期间会使用许多存储器地址和寄存器，并支持通过 DCSM 区域 1（Z1）和区域 2（Z2）寄存器配置引导。独立引导流程中可由用户配置的 DCSM OTP 存储位置只能编程一次。寄存器配置详见第 2.1 节。

在 DCSM 中，BOOTPIN-CONFIG 映射到 GPREG1，BOOTDEF-LOW/BOOTDEF-HIGH 分别映射到 GPREG3/GPREG4。表 2-7 列出了这些位置。

第5.4 节和第 5.5 节详细说明了如何使用 On-Chip Flash 工具和 Flash API 分别对应以下示例用例。SysConfig 也可用于对 DCSM OTP 编程，并提供直观的用户界面（GUI）[8]。

注

表 2-7 中的寄存器地址适用于 F280015x 器件系列。器件专属信息可在技术参考手册（TRM）的 *Boot ROM* 章的 *Boot ROM Registers* 表中找到。

表 2-7. F280015x Boot ROM 寄存器

Boot Flow	Register Name	Boot ROM Name	Register Address	User OTP Address
Standalone (Using Z1)	Z1-GPREG1	Z1-OTP-BOOTPIN-CONFIG	0x0005 F008	0x0007 8008
	Z1-GPREG2	Z1-OTP-BOOT-GPREG2	0x0005 F00A	0x0007 800A
	Z1-GPREG3	Z1-OTP-BOOTDEF-LOW	0x0005 F00C	0x0007 800C
	Z1-GPREG4	Z1-OTP-BOOTDEF-HIGH	0x0005 F00E	0x0007 800E
Standalone (Using Z2)	Z2-GPREG1	Z2-OTP-BOOTPIN-CONFIG	0x0005 F088	0x0007 8208
	Z2-GPREG2	Z2-OTP-BOOT-GPREG2	0x0005 F08A	0x0007 820A
	Z2-GPREG3	Z2-OTP-BOOTDEF-LOW	0x0005 F08C	0x0007 820C
	Z2-GPREG4	Z2-OTP-BOOTDEF-HIGH	0x0005 F08E	0x0007 820E

2.1.4 CPU2 Boot Flow

注：本节以 F28P65x 器件系列为例。器件专属信息请参阅器件专用技术参考手册（TRM）的 *Boot ROM* 章。

虽然 CPU1 可根据引导引脚配置选择不同的引导模式，但 CPU2 必须由 CPU1 通过处理器间通信（IPC）模块引导 [9]。CPU1 应用程序会配置引导模式
并通过 IPCBOOTMODE 寄存器控制 CPU2 何时解除复位并开始引导。

无论复位来源为何，每次复位时都必须由 CPU1 设置 IPC 标志，以确认 IPCBOOTMODE 寄存器内容有效并继续引导流程。CPU2 会在引导期间确认并清除该标志。

版权所有 © 2025 Texas Instruments Incorporated

CPU2 的引导序列大致如下：

1. CPU2 启动后处于以下状态之一：a. 保持复位；b
. 处于 Wait 引导模式并等待 IPC 标志

2. CPU1 应用程序配置 CPU1TOCPU2IPCBOOTMODE 寄存器 3. CPU1 设置 CPU1TOCPU2IPCFLG0，以确认 CPU 1TOCPU2IPCBOOTMODE 内容有效 4. 如果 CPU2 处于复位状态，CPU1 应用程序会释放 CPU2 5. CPU2 在引导期间确认并清除 IPC 标志 6. CPU2 Boot ROM 执行 CPU1TOCPU2IPCBOOTMODE 中指定的引导模式

F28P65x TRM 的 *IPCBOOTMODE Details* 部分说明了用于引导 CPU2 的 IPCBOOTMODE 寄存器位字段配置和相关要求。表 2-8 详细说明该寄存器如何配置以用于
F28P65x 器件。

与 CPU1 的 BOOTPIN-CONFIG 和 BOOTDEF 寄存器类似，CPU1TOCPU2IPCBOOTMODE 的配置 由如下内容组成：

1. 高 8-bits 包含用于标记有效性的密钥（0x5A） 2. 低 8-bits 用于设置 CPU2 的引导模式 3. Bits 16-19 指定在调用“从 IPC 消息 RAM 复制并引导至 M1RAM”模式时，要从 CPU1TOCPU2MSGRAM1 复制到 CPU2 M1RAM 的字数

表 2-8. CPU1TOCPU2IPCBOOTMODE 寄存器详细信息

位段	名称	有效值	说明
31:24	密钥	0x5A	必须设置密钥，寄存器才视为有效。
23:20	保留	-	保留
19:16	IPC Message RAM 复制长度	0x0 = 0 字（未用） 0x1 = 100 字 0x2 = 200 字 ... 0x9 = 900 字 0xA = 1000 字	设置数据长度（单位：字），用于“从 IPC 消息 RAM 复制并引导至 M1RAM”模式。该值表示需复制的数据字数，源为 CPU1TOCPU2MSGRAM1，复制到 CPU2 M1RAM。 未使用此引导模式时，将值设为 0x0。
15:8	保留	-	保留
7:0	CPU2 引导模式	0x00 = 无/等待引导 0x01 = IPC Message RAM 复制并引导至 M1RAM 0x03 = 闪存引导选项 0（扇区 0） 0x05 = 引导至 MORAM 0x0A = 安全闪存引导选项 0（扇区 0） 0x0B = 引导至用户 OTP 0x23 = 闪存引导选项 1（扇区 4） 0x2A = 安全闪存引导选项 1（扇区 4） 0x43 = 闪存引导选项 2（扇区 8） 0x4A = 安全闪存引导选项 2（扇区 8） 0x63 = 闪存引导选项 3（扇区 13） 0x6A = 安全闪存引导选项 3（扇区 13）	设置 CPU2 的引导模式。

版权所有 © 2025 Texas Instruments Incorporated

2.2 仿真引导

如果器件连接了 JTAG 调试器，器件就会进入仿真引导模式。与独立引导模式类似，通过对位于 RAM 中的仿真引导寄存器编程，也可访问默认引导表之外的引导选项，从而允许用户反复配置引导。这样，用户便可通过调试器测试引导配置并查看 Boot ROM 状态（见第 5.3 节）。

Note

TI highly recommends using emulation mode to debug and verify correctness of the device boot configurations before attempting to program the OTP, as you can only program the OTP once.

- EMU-BOOTPIN-CONFIG 是 Z1-OTP-BOOTPIN-CONFIG/Z2-OTP-BOOTPIN-CONFIG 的仿真版本，可在不写入 OTP 的情况下编程，以试验不同的引导模式。
- EMU-BOOTDEF-LOW/EMU-BOOTDEF-HIGH 是 Z1-OTP-BOOTDEF-LOW/Z1-OTP-BOOTDEF-HIGH 的仿真版本。

可在 RAM 中反复写入表 2-9 所列位置处的仿真寄存器；请使用 CCS 中的存储器浏览器：

表 2-9. 仿真引导寄存器位置

Boot ROM Name	Register Address
EMU-BOOTPIN-CONFIG	0x0000 0D00
EMU-GPREG2	0x0000 0D02
EMU-BOOTDEF-LOW	0x0000 0D04
EMU-BOOTDEF-HIGH	0x0000 0D06

仿真引导模式启动时，系统会检查 EMU-BOOTPIN-CONFIG 和 EMU-BOOTDEF 位置，并验证 EMU-BOOTPIN-CONFIG-KEY。

- 如果 EMU-BOOTPIN-CONFIG-KEY 等于 0xA5，CPU 就会模拟独立引导，本质上是使用已编程的 OTP 定义。
- 如果 EMU-BOOTPIN-CONFIG-KEY 等于 0x5A，则会使用指定的 BMSP 解码仿真 BOOTDEF 选项，并执行所选的引导模式。

版权所有 © 2025 Texas Instruments Incorporated

3 闪存编程

尝试对器件编程之前，请先了解 C2000 器件非易失性存储器的工作方式。C2000 器件的闪存可供用户轻松擦除并重新编程；即使断电，数据也不会丢失。擦除操作会将给定扇区中的所有位设置为 *1*，而编程操作则会选择性地将位清零为 *0*。

开发期间，可使用 Code Composer Studio™（CCS）[1] 将应用程序可执行文件编程到闪存中。CCS 识别到应用程序代码映射在闪存中后，会自动调用片上闪存插件，将可执行文件加载到闪存。默认情况下，该插件会先擦除闪存，为可执行文件生成 ECC，然后将应用程序编程到闪存并进行验证。

片上闪存插件 GUI　可在连接到目标 CPU 内核后使用，　在 CCS 中的位置如下：

- 对于 CCS v12，*Tools > On-Chip Flash*

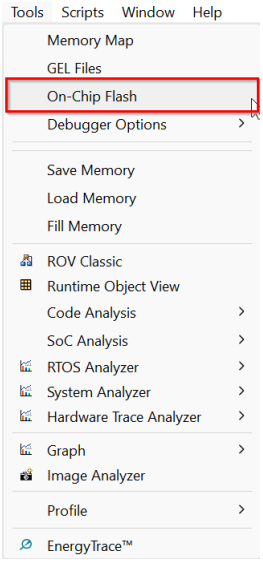

图 3-1. CCS v12 中片上闪存工具的位置

- 对于 CCS v20，在 *Debug* 视图中右键单击目标 CPU，然后依次选择 *Properties > Flash Settings* 。(*Flash Settings* 位于 *Categories* 下拉菜单中)

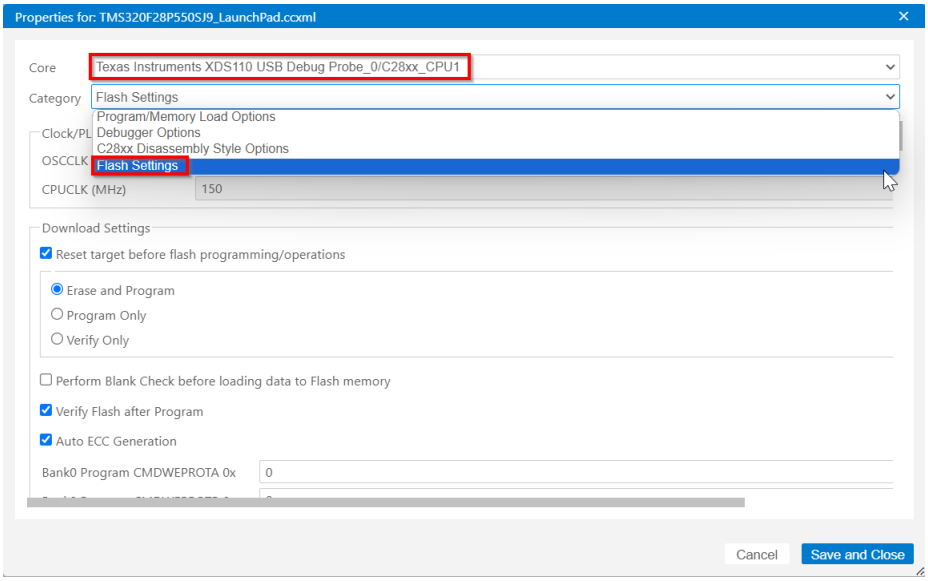

图 3-2. CCS v20 中片上闪存工具的位置

版权所有 © 2025 Texas Instruments Incorporated

TI 还提供 UniFlash[2]，可用于应用程序闪存编程。UniFlash 是一款独立的、基于 JTAG 的闪存编程工具，由于调试支持较少，其占用空间小于 CCS。不过，UniFlash 提供了与 CCS 片上闪存插件相同的全部 GUI 操作。

3.1 闪存 API

C2000 器件上的闪存操作均由 CPU 执行。操作算法会先加载到 RAM，再由 CPU 执行，以完成相应的闪存操作。例如，使用 CCS 擦除或编程 C2000 器件的闪存时，CCS 会通过 JTAG 将闪存算法加载到 RAM，然后由 CPU 执行这些算法。

所有闪存操作都通过闪存应用编程接口（API）完成。器件专用库及参考指南可在 C2000Ware[3] 的 libraries/flash_api 目录中找到。若应用需要在运行时擦除或编程闪存，可链接 Flash API 库来执行闪存编程。

但是，不建议调用 Flash API 的应用程序从同一闪存区运行，因为在执行代码的同时擦除或编程闪存会引入竞争条件，并导致未定义行为。因此，Flash API 必须在 RAM 中执行；如果同一内核具有其他闪存区，也可以在另一个闪存区中执行。

可在链接器命令文件中将 Flash API 分配到 .ti.ramfunc 段，并指定闪存加载地址和 RAM 运行地址，然后在执行前于 main 函数中将这些函数复制到 RAM。有关 C2000 器件闪存编程的详细说明，请参阅 [4] 和 [5]。

注意

在多核器件上，一个 CPU 无法访问分配给另一个 CPU 的闪存区。例如，CPU2 的闪存区只能通过在 CPU2 RAM 中执行 Flash API 来编程。

3.2 闪存内核

由于 Boot ROM 中的引导加载程序只能将代码加载到 RAM，闪存内核便在 ROM 与闪存之间建立了连接，使设备能够通过各种通信协议（SCI、CAN、I2C 等）进行现场固件升级。编程闪存时，可先使用 ROM 引导加载程序将闪存内核下载到 RAM，再在 RAM 中运行该内核，通过 Flash API 将应用程序下载并写入闪存。

接收任何应用程序数据之前，闪存内核会先擦除器件闪存，为编程做好准备。主机开始发送应用程序代码后，内核会使用缓冲区存放连续接收到的代码块。缓冲区写满或检测到新的非连续数据块时，内核就会将缓冲区中的代码写入闪存。接收并写入全部应用程序后，闪存内核会跳转到应用程序入口点。

TI 基于 Boot ROM 源代码开发了用于将代码从 RAM 加载到闪存的闪存内核，这些内核位于 C2000Ware 的 *examples* 文件夹中。用户也可采用类似的开发流程，为特定应用实现自定义引导加载程序。器件引导流程所用的 Boot ROM 源代码（包括外设引导加载程序）可在 C2000Ware 中查看，位置如下：

- *C2000Ware_x_xx_xx_xx > libraries > boot_rom > DEVICE_FAMILY > REV# > rom_sources > DEVICE_FAMILY_ROM > bootROM > source*

对于具有多个闪存区的器件，可修改闪存内核项目，使其从闪存而非 RAM 执行，从而对另一个闪存区编程。这样，用户只需跳转到位于闪存中的闪存内核，无需使用 Boot ROM 中的引导加载程序。有关实现方法的详细信息，请参阅第 5.2 节。

版权所有 © 2025 Texas Instruments Incorporated

注意

在多核器件 (即 F2838x、F2837xD 或 F28P65x) 上，CPU1 和 CPU2 的内核项目可利用 CPU1 中的引导加载程序下载经过修改的引导加载程序（即闪存内核），再由该内核下载 CPU2 的应用程序映像。

CPU1 的操作完成后，可将 CPU2 内核写入共享消息 RAM (CPU1TOCPU2MSGRAM)，并写入一条指令，使 CPU2 在引导过程中跳转到 CPU2 IPC 消息复制目标 RAM（M1RAM）中的 CPU2 入口点。该跳转指令写入 M1RAM 且 CPU2 引导序列完成后，CPU2 会从 M1RAM 开始执行，并跳转到 CPU2 内核入口点。CPU1 内核会等待 CPU2 内核命令执行完毕后再继续。

有关该器件 专用技术参考手册，请参阅其中关于 IPC 协议的更多详细信息。

总的来说，使用闪存内核的流程如下：

1. 复位器件，并选择所需的引导模式 2. 通过 Boot ROM 中的引导加载程序，将闪存内核从主机传输到器件 3. ROM 引导加载程序完成后，由闪存内核接管控制权 4. 内核擦除控制器闪存中的旧应用程序代码 5. 内核与主机建立连接，并使用所需的外设通信协议接收新的应用程序代码 6. 内核将接收到的新应用程序代码写入闪存，然后将控制权交给应用程序 7. 执行从主机接收的新代码

版权所有 © 2025 Texas Instruments Incorporated

4 将代码引导加载到闪存

器件开始引导时，会判断闪存中是否已有需要执行的固件，还是需要通过 ROM 加载器加载代码。系统会检查 BMSP；BMSP 可能由用户在 OTP 中定义，也可能由仿真寄存器定义，或者遵循出厂默认引导配置。

但是，如果选择了从外部主机加载代码的引导加载程序，则必须按照特定流程对应用程序映像进行格式化并传输，才能将其正确编程到 RAM。随后，可使用闪存内核连接闪存与 RAM，并通过 Flash API 擦除和编程闪存。

本节将介绍如何针对最常用的引导模式（Flash、SCI、CAN、CAN-FD 和 USB），通过完整流程将应用程序编程到闪存和/或从闪存执行，流程涵盖从器件引导配置到闪存内核执行。

4.1 C2000 十六进制实用程序

ROM 加载器要求以数据流和引导表的形式提供数据。所有 ROM 加载器均使用相同的数据结构，器件专用 TRM 的 ***Bootloader Data Stream Structure*** 部分对此有详细说明。用户可使用 TI C2000 编译器随附的 hex2000 实用程序，轻松生成这种格式的应用程序文件。

也可在 Code Composer Studio 的构建流程中添加一个构建后步骤来生成此文件格式，如图 4-1 所示。在项目属性中启用 ***C2000 Hex Utility*** 并选择所需的转换选项后，还可通过 GUI 配置 hex2000 实用程序。

对于 C2000 ROM 引导加载程序，需要在 CCS Build (Project Properties > CCS Build > Steps > Post-build steps) 中的构建后步骤添加以下命令。该命令适用于编程到片上闪存的固件项目，以及加载到 RAM 的闪存内核项目：

SCI、CAN、CAN-FD 引导加载：

```
"${CG_TOOL_HEX}" "${BuildArtifactFileName}" -boot -sci8 -a -o "${BuildArtifactFileBaseName}.txt"
```

USB 引导加载：

```
"${CG_TOOL_HEX}" "${BuildArtifactFileName}" -boot -b -o "${BuildArtifactFileBaseName}.dat"
```

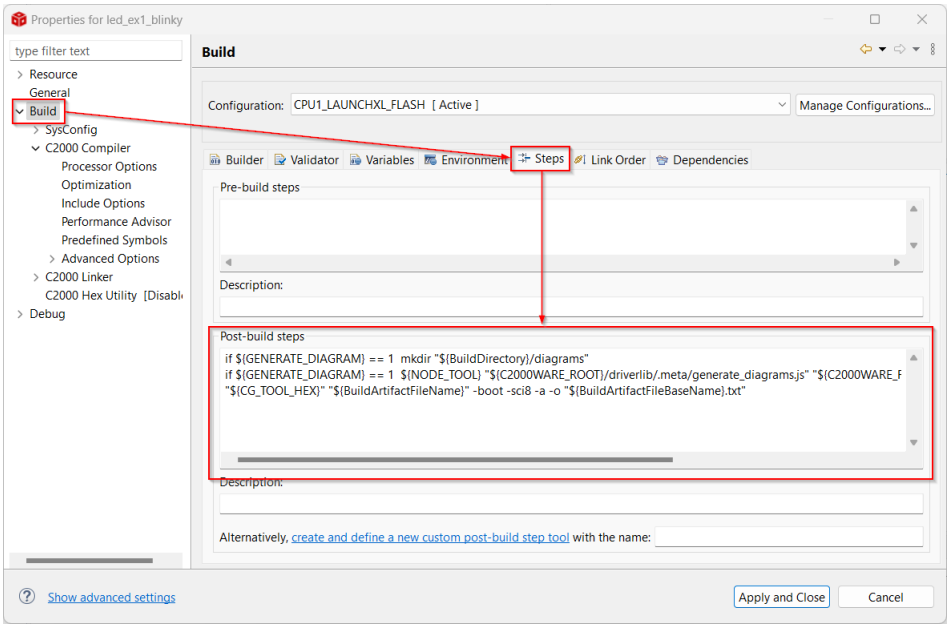

图 4-1. 添加构建后步骤以调用十六进制实用程序

版权所有 © 2025 Texas Instruments Incorporated

十六进制实用程序支持生成 SCI、SPI、I2C、CAN 和并行 I/O 加载器所需的引导表。该实用程序会向文件中添加密钥值、保留位、入口点、地址、块起始地址、块长度和终止值等必要信息。引导表内容会因引导模式及十六进制转换实用程序的选项而略有不同。主机所需的实际文件格式（ASCII、二进制、十六进制等）因具体应用而异，可能还需要进行额外转换。

关于生成引导表所用 hex2000 选项的详细说明，请参阅《TMS320C28x Assembly Language Tools User's Guide》[11]。

4.2 常见引导模式

本节完整介绍如何针对最常用的引导模式（Flash、SCI、CAN、CAN-FD 和 USB），将应用程序编程到闪存并从闪存执行的整个流程。

注意

本节主要介绍在仿真器（JTAG）已连接且未使用 BMSP（零引脚引导）时的引导执行路径。仿真引导寄存器位于 RAM 中，其格式与 OTP 寄存器相同，因此可参考第 5.4 节或第 5.5 节，将仿真引导流程应用于采用不同 BMSP 配置的独立引导。

4.2.1 Boot to Flash

注意

虽然以下步骤是在 F2800157 LaunchPad 上执行的，但这一通用流程也适用于支持自定义 BMSP 和引导定义表的其他 C2000 器件（表 2-3 中列出的所有器件）。有关目标器件的详细信息，请参阅相应的器件专用 TRM。

如果用户需要引导执行已编程到片上闪存中的代码，可使用默认 BMSP，将程序引导到闪存入口地址 0x0008 0000；也可配置 BOOTPIN-CONFIG 和 BOOTDEF 寄存器，将程序引导到指定的闪存地址。

注意

某些器件支持安全闪存引导选项，可在应用程序从闪存引导时，先通过额外的引导代码身份验证增加一层安全保护。有关启用安全引导的一般流程，请参阅 *Secure BOOT on C2000 Device* [13]。用户可在器件专用 TRM 中确认该器件是否支持安全闪存引导。

用于配置闪存引导的默认 BMSP 见 TMS320F280015x Real-Time Microcontrollers 数据手册。如果将 GPIO24 和 GPIO32 均设为 1，Boot ROM 就会跳转到闪存入口地址 0x0008 0000，无需配置器件寄存器。

如果需要引导到其他闪存扇区，则必须针对相应引导选项配置 BOOTCONFIG 和 BOOTDEF 寄存器。请参阅数据手册中的 *GPIO Assignments* 部分，了解应配置哪个引导选项才能跳转到目标闪存入口地址。以下步骤说明如何模拟引导到闪存入口地址 0x0009 0000，例如使用引导选项 0x63。

注意

通过链接器命令文件，可将应用程序加载到不同的闪存位置。有关如何将代码加载到特定闪存地址的详细信息，请参阅 *Compiler Tools User Manual* 关于 Linker Command Files 的说明 [12]。

1. 在 CCS 中打开一个工作区。2. 选择 View > Target Configurations。

版权所有 © 2025 Texas Instruments Incorporated

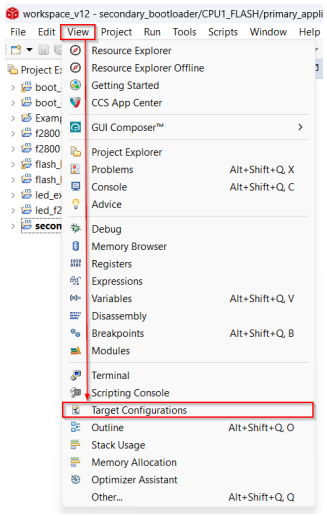

图 4-2. 在 CCS 中打开目标配置菜单

3. 可将该器件的项目导入 CCS，并使用该项目连接器件；也可从 C2000Ware 复制目标配置文件（.ccxml），路径为 (C2000Ware_x_xx_xx_xx > device_support > DEVICE_FAMILY > common > targetConfigs)，然后将其复制到 ***User Defined*** target configurations。

a. 找到器件的目标配置文件，然后右键单击并手动启动。

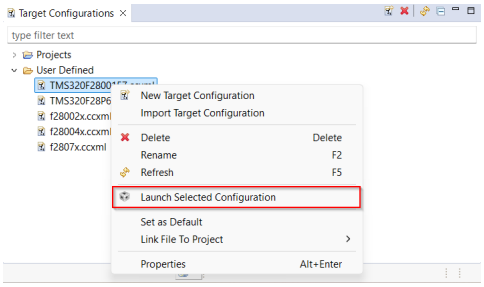

图 4-3. 在 CCS 中启动目标配置

4. CCS 打开调试窗口后，选择目标 CPU 并连接到目标器件。

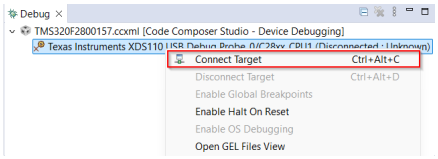

图 4-4. 连接到目标内核

5. 如果弹出窗口提示 Boot ROM 中存在断点但没有可用的调试信息，或断点位于程序代码之外，请按照第 5.3 节调试 Boot ROM。6. 加载符号后，选择 View > Memory Browser 打开存储器浏览器。

版权所有 © 2025 Texas Instruments Incorporated

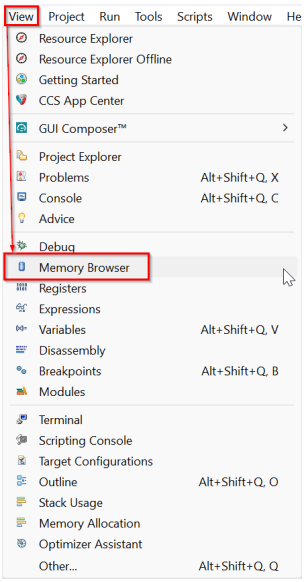

图 4-5. 在 CC S 中打开存储器浏览器

7. 在存储器浏览器选项卡中，转到地址 0xD00。地址 0xD00 用于指定带有效性密钥（EMU-BOOTPIN-CONFIG）的 BMSP；地址 0xD04-0xD05 用于指定引导定义（EMU-BOOTDEF-LOW）。

8. 目标是配置零引脚引导到闪存地址 0x0009 0000，因此必须禁用所有 BMSP，并将 EMU-BOOTDEF-LOW 的最低索引设置为 0x63。如果将引导选项写入
EMU-BOOTDEF-LOW 中的其他条目，则不会选择预期的引导模式。
a. 将 0xD00-0xD01（EMU-BOOTPIN-CONFIG）设为 0x5AFF FFFF
。b. 将 0xD04（EMU-BOOTDEF-LOW）设为 0x0063。

注意

零引脚引导是指器件自动引导到 BOOTDEF 表中定义的第一项。为此需禁用所有 BMSP，使器件仅考虑一个引导选项。

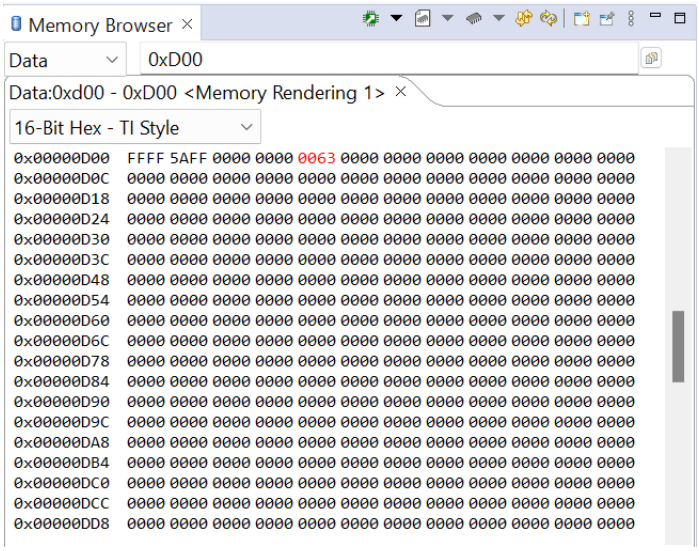

图 4-6. 模拟零引脚引导到闪存 (0x0009 0000)

9. 复位 CPU 并执行外部复位（XRSn），然后单击 ***Resume*** 开始引导序列。10. 随后，器件复位时会按照引导选项 0x63 的指定，引导到闪存地址 0x0009 0000。

版权所有 © 2025 Texas Instruments Incorporated

4.2.2 SCI Boot

注意

虽然这些步骤是在 F2800157 LaunchPad 上执行的，但这一通用流程也适用于支持自定义 BMSP 和引导定义表的其他 C2000 器件（表 2-3 中列出的所有器件）。有关目标器件的详细信息，请参阅相应的器件专用 TRM。

SCI 闪存内核基于 ROM 引导加载程序，与主机 PC 应用程序通信。该应用程序由 C2000Ware 提供 (C2000Ware_x_xx_xx_xx > utilities > flash_programmers > serial_flash_programmer)，并向主机反馈数据包接收情况，以及所执行命令的完成状态。

注意

本节介绍 CPU1 SCI 引导加载。有关 SCI 内核命令和功能的详细说明，以及如何使用 CPU2 或通信管理器（CM）SCI 引导加载程序，请参阅
Serial Flash Programming of C2000 Microcontrollers application note [12]。

C2000Ware 在相应器件的 examples 目录中提供了 CCS 闪存内核源文件和项目。这些项目包含一个构建后步骤，可将编译和链接生成的 .out 文件转换为 SCI ROM 引导加载程序所需的引导十六进制格式，并以项目名称加 .txt 扩展名保存。

1. 在 C2000Ware 中找到目标器件对应的 SCI 闪存内核项目，并将其导入 CCS。

Device	Build Configurations	Location
F2802x	RAM	C2000Ware_x_xx_xx_xx > device_support > f2802x > examples > structs > f28027_flash_kernel
F2803x	RAM	C2000Ware_x_xx_xx_xx > device_support > f2803x > examples > c28 > f2803x_flash_kernel
F2805x	RAM	C2000Ware_x_xx_xx_xx > device_support > f2805x > examples > c28 > f28055_flash_kernel
F2806x	RAM	C2000Ware_x_xx_xx_xx > device_support > f2806x > examples > c28 > f28069_sci_flash_kernel
F2807x	RAM	C2000Ware_x_xx_xx_xx > device_support> f2807x > examples > cpu1 > F2807x_sci_flash_kernel
F2833x	RAM	C2000Ware_x_xx_xx_xx > device_support > f2833x > examples > f28335_flash_kernel
F2837xS	RAM	C2000Ware_x_xx_xx_xx > device_support > f2837xs > examples > cpu1 > F2837xS_sci_flash_kernel > cpu01
F2837xD	RAM	C2000Ware_x_xx_xx_xx > device_support > f2837xd > examples > dual > F2837xD_sci_flash_kernels
F28004x	RAM, Flash with LDFU, Flash without LDFU	C2000Ware_x_xx_xx_xx > driverlib > f28004x > examples > flash, select flashapi_ex2_sci_kernel
F2838x	RAM	CPU1-CPU2 C2000Ware_x_x_xx_xx > driverlib > f2838x>examples>c28x_dual>flash_kernel CPU1-CM C2000Ware_x_x_xx_xx > driverlib > f2838x>examples>c28x_cm>flash_kernel
F28002x	RAM, Flash with LDFU	C2000Ware_x_xx_xx_xx > driverlib > f28002x > examples > flash, select flash_kernel_ex3_sci_flash_kernel
F28003x	RAM, Flash with LDFU	C2000Ware_x_xx_xx_xx > driverlib > f28003x > examples > flash, select flash_kernel_ex3_sci_flash_kernel
F280013x	RAM	C2000Ware_x_xx_xx_xx > driverlib > f280013x > examples > flash, select flash_kernel_ex3_sci_flash_kernel

版权所有 © 2025 Texas Instruments Incorporated

Device	Build Configurations	Location
F280015x	RAM	C2000Ware_x_xx_xx_xx > driverlib > f280015x > examples > flash, select flash_kernel_ex3_sci_flash_kernel
F28P65x	RAM	C2000Ware_x_xx_xx_xx > driverlib > f28p65x > examples > c28x_dual > flash_kernel
F28P55x	RAM	C2000Ware_x_xx_xx_xx > driverlib > f28p55x > examples > flash, select f28p55x_flash_ex3_sci_flash_kernel

2. 确认 SCI 闪存内核项目的活动构建目标配置设为 ***RAM***，因为内核必须链接为在 RAM 中执行。

3. 构建内核项目。这些项目包含一个构建后步骤，可将编译和链接生成的 .out 文件转换为 SCI ROM 引导加载程序所需的引导十六进制格式，并以示例名称加 txt 扩展名保存。

   a. 如果未生成 txt 输出文件，请按照第 4.1 节检查是否已定义正确的构建后步骤来生成十六进制文件。

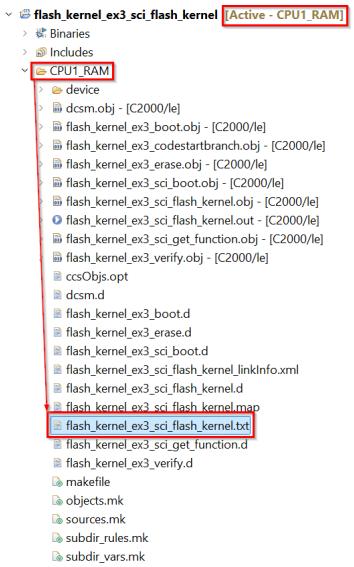

图 4-7. 查找转换后的 SCI 内核输出文件

4. 对将由内核加载到闪存中的固件代码重复构建流程。a. 确认活动构建目标配置设为 ***Flash***。b. 确认已定义用于生成 txt 文件的正确构建后步骤。c. 构建固件项目。

在 CCS 中构建内核和固件项目后，应正确设置器件硬件，以便与运行 serial_flash_programmer 可执行文件的主机 PC 通信。该可执行文件由 C2000Ware 提供。首先要正确配置引导模式选择引脚，使器件进入 SCI 引导模式。如果需要从外部主机将代码加载到片上闪存，可通过默认 BMSP 或 BOOTPIN-CONFIG 和 BOOTDEF 寄存器配置 SCI 引导。

用于启用 SCI 引导的默认 BMSP 见 TMS320F280015x Real-Time Microcontrollers data sheet。如果将 GPIO24 设为 0、GPIO32 设为 1，Boot ROM 就会跳转到 SCI 引导加载程序，并将 SCIRXDA 连接到 GPIO28、将 SCITXDA 连接到 GPIO29，无需编程器件寄存器。

但是，如果用户希望使用不同 GPIO 分配方式灵活配置 SCI 引导，则必须针对相应引导选项配置 OTP 或仿真 BOOTCONFIG 和 BOOTDEF 寄存器。请参阅数据手册中的 ***GPIO Assignments*** 部分，了解符合 GPIO 要求的 SCI 引导选项。

使用 serial_flash_programmer 可执行文件时，需要将 SCI 引导加载程序对应的 GPIO Rx 和 Tx 引脚连接到主机 PC 的 COM 端口。通常需要使用收发器，将 PC 的虚拟

版权所有 © 2025 Texas Instruments Incorporated

COM 端口转换为可连接到器件 GPIO 引脚的信号。在 controlCARD 或 LaunchPad 等系统中，通常使用 FTDI 芯片将 SCI 通信 GPIO 引脚连接到 USB 虚拟 COM 端口。

对于 F2800157 LaunchPad，PC 必须连接到 LaunchPad 上的 USB，并使用器件上的 UART 路由将 GPIO 引脚连接到 XDS110 COM 端口。UART 路由原理图位于 C2000Ware_x_xx_xx_xx > boards > (LaunchPads or controlCARDs) > DEVICE_NAME > Rev# > documentation。对于 F2800157，默认 SCI_SEL 设置会将 SCIRXDA 内部连接到 GPIO28、将 SCITXDA 内部连接到 GPIO29，因此必须配置引导选项 0x01。

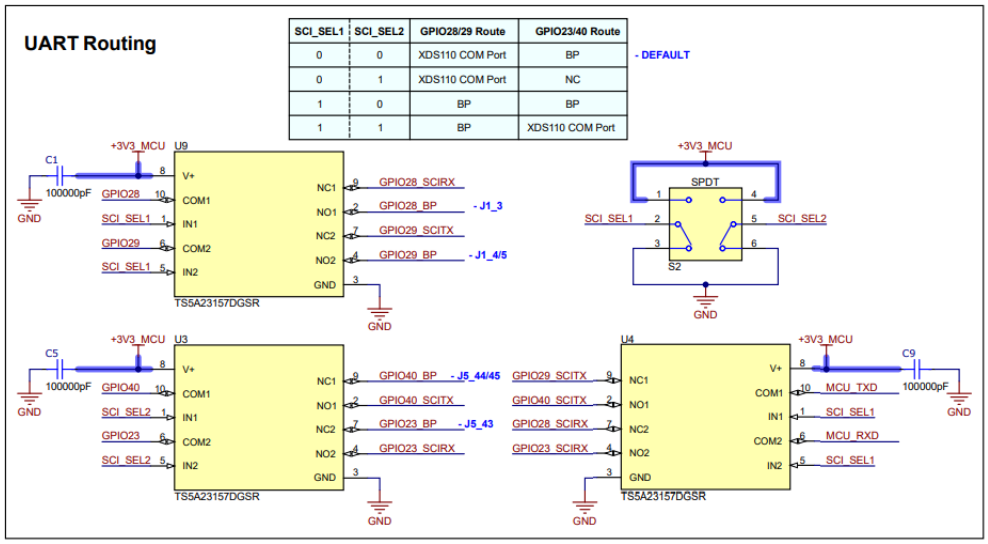

图 4-8. F2800157 LaunchPad UART 路由原理图

用户也可以使用跳线，将 XDS110 COM 端口从外部连接到 GPIO BoosterPack™（BP）接头。如果选择了其他 SCI GPIO 分配方式且仍需连接 XDS COM 端口，这种方法会很有帮助。以下步骤说明如何通过外部跳线，将 XDS110 COM 端口连接到 F2800157 LaunchPad 上的 GPIO28 或 GPIO29。

1. 确保 UART 路由设置为连接到 BoosterPack™（BP），以供 GPIO28 和 GPIO29 使用（SCI_Sel1 = 1）；不要将其内部连接到 XDS110 COM 端口。这样，SCI-A 信号便会输出到 BP 接头引脚。

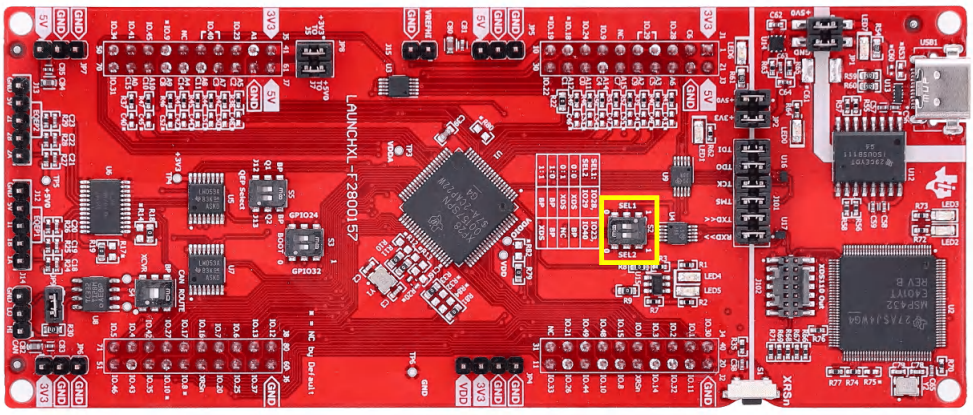

图 4-9. 将 SCI TX 或 RX（GPIO28 或 GPIO29）信号路由到 BoosterPack（BP）引脚

版权所有 © 2025 Texas Instruments Incorporated

2. 下　J101 接头上 TXD 和 RXD 引脚的跳线。请注意，XDS RXD 引脚（靠近 XDS 电路）连接到 MCU TXD，且该引脚位于靠近 XDS 电路的一侧。类似地，XDS TXD 连接到 MCU RXD。

3. 按照电路板原理图，将 XDS TXD 连接到 GPIO28（SCI-A RX），并将 XDS RXD 连接到 GPIO 29（SCI-A TXD）；具体连接方式参见电路板原理图中的 *XDS110 Target Interface* 部分。

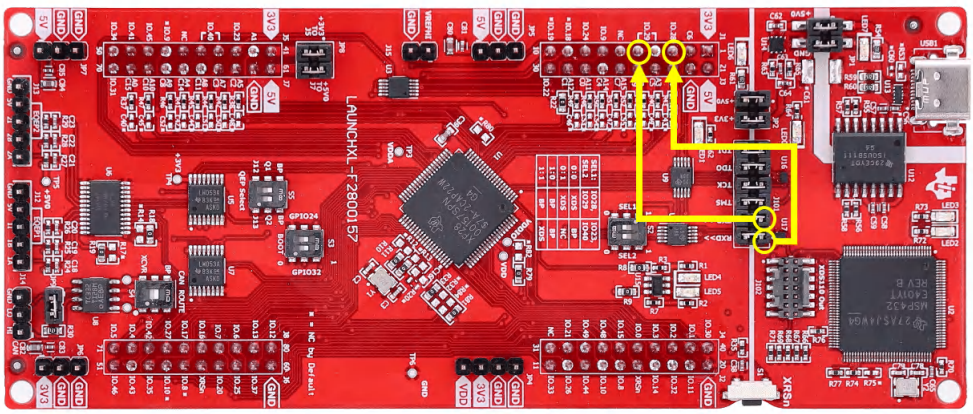

图 4-10. 将 XDS TX、RX 连接到 SCI TX、RX GPIO

现在需要将器件配置为模拟零引脚 SCI 引导，使用引导选项 0x01，并将 SCIRXA = GPIO28、SCITXA = GPIO29。

1. 在 CCS 中打开一个工作区。2. 选择 View > Target Configurations。

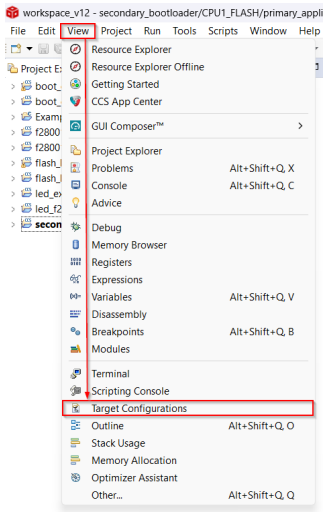

图 4-11. 在 CCS 中打开目标配置菜单

3. 可将该器件的项目导入 CCS，并使用该项目连接器件；也可从 C2000Ware 复制目标配置文件（.ccxml），路径为 (C2000Ware_x_xx_xx_xx > device_support > DEVICE_FAMILY　> common > targetConfigs)，然后将其复制到 *User Defined* target configurations。

a. 找到器件的目标配置文件，然后右键单击并手动启动。

版权所有 © 2025 Texas Instruments Incorporated

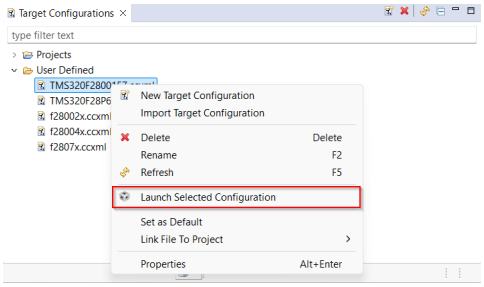

图 4-12. 在 CCS 中启动目标配置

4. CCS 打开调试窗口后，选择目标 CPU 并连接到目标器件。

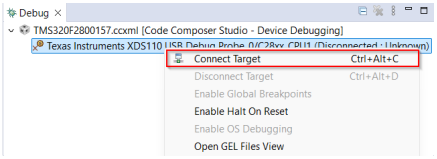

图 4-13. 连接到目标内核

5. 如果弹出窗口提示 Boot ROM 中存在断点但没有可用的调试信息，或断点位于程序代码之外，请按照第 5.3 节调试 Boot ROM。6. 加载符号后，选择 View > Memory Browser 打开存储器浏览器。

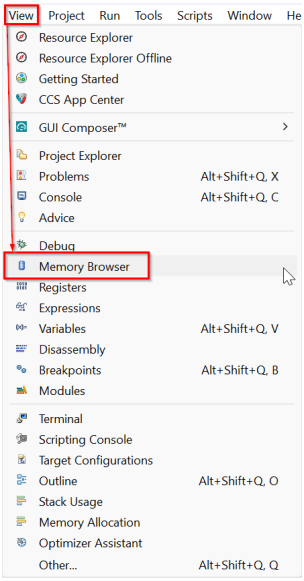

图 4-14. 转到存储器浏览器

7. 在存储器浏览器选项卡中，转到地址 0xD00。请注意，地址 0xD00 用于指定带有效性密钥（EMU-BOOTPIN-CONFIG）的 BMSP；地址 0xD04-0xD05 用于指定引导定义（EMU-BOOTDEF-LOW）。

8. 目标是配置零引脚 SCI 引导，使 SCIRXDA 连接到 GPIO28、SCITXDA 连接到 GPIO29，因此必须将所有 BMSP，并将 EMU-BOOTDEF-LOW 的最低索引设置为 0x01。如果将引导选项写入 EMU-BOOTDEF-LOW 中的其他条目，则不会选择预期的引导模式。

a. 将 0xD00-0xD01（EMU-BOOTPIN-CONFIG）设为 0x5AFF FFFF
。b. 将 0xD04（EMU-BOOTDEF-LOW）设为 0x0001。

版权所有 © 2025 Texas Instruments Incorporated

注意

零引脚引导是指器件自动引导到 BOOTDEF 表中定义的第一项。为此需禁用所有 BMSP，使器件仅考虑一个引导选项。

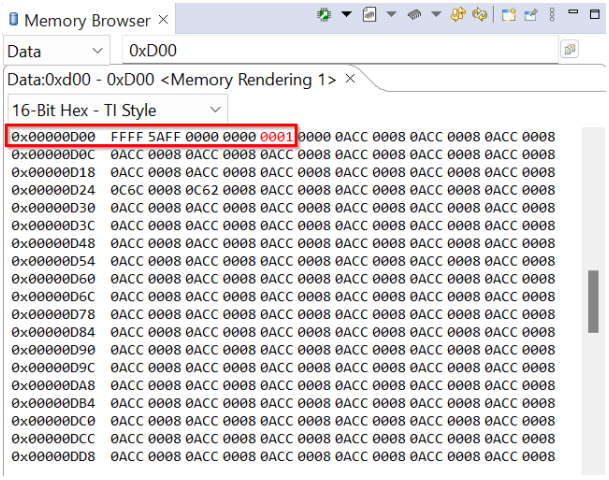

图 4-15. 模拟零引脚 SCI 引导：SCIRXDA 连接到 GPIO28，SCITXDA 连接到 GPIO29

9. 复位 CPU 并执行外部复位（XRSn），然后单击 ***Resume*** 开始引导序列。10. 如果 Boot ROM 中出现断点，但没有可用的调试信息，或断点位于程序代码之外，请按照第 5.3 节加载 Boot ROM 符号。随后确认器件已进入 SCI 自动波特率锁定状态。

随后，ROM 中的 SCI 引导加载程序（引脚分配由引导选项 0x01 指定）开始执行，并等待主机发送 ***A*** 字符，以确定通信波特率并完成自动波特率锁定。此时器件已准备好从主机接收代码。

命令行 PC 实用程序是一种便于集成到生产线编程等脚本环境中的编程方案。该程序使用 Microsoft Visual Studio® 和 C++ 编写。项目和源代码位于 C2000Ware (C2000Ware_x_xx_xx_xx > utilities > flash_programmers > serial_flash_programmer)。

使用此工具编程 C2000 器件前，请确认目标板已复位、处于上述配置的 SCI 引导模式，并已连接到 PC COM 端口。下面介绍该工具在单核 SCI 引导时的命令行用法，其中 -d、-k、-a 和 -p 为必需参数。若未指定波特率，则默认设为 9600。有关这些参数的更多信息，请参阅 [12]。

```
serial_flash_programmer.exe –d DEVICE -k KERNEL_FILE -a APPLICATION_FILE -p COM# -b BAUDRATE -v
```

1. 打开设备管理器，选择 > Ports (COM & LP)，查找 XDS110 UART COM 端口。

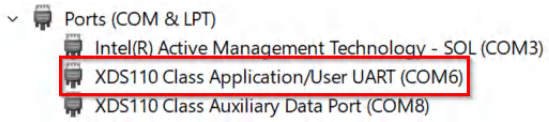

图 4-16. 在设备管理器中查找 XDS COM 端口

2. 打开包含已编译 serial_flash_programmer 可执行文件的文件夹 (C2000Ware_x_xx_xx_xx > utilities > flash_programmers > serial_flash_programmer)。运行 serial_flash_programmer.exe，并使用以下命令：

```
serial_flash_programmer.exe –d DEVICE_NAME –k <path_to_kernel_hex> -a <path_to_application_hex>  -p COM# -v
```


版权所有 © 2025 Texas Instruments Incorporated

注意

闪存内核和闪存应用程序均必须采用第 4.1 节介绍的 SCI8 引导格式。

此工具会自动连接器件、执行自动波特率锁定，并将 CPU1 内核下载到 RAM 并执行。此时 CPU1 内核正在运行，等待主机发送数据包。

1. serial_flash_programmer 会在屏幕上显示选项，供用户选择要发送给器件内核的命令。选择 *1-DFU CPU1* 将应用程序写入 CPU1。由于原命令中已指定应用程序文件，此时无需提供其他信息。

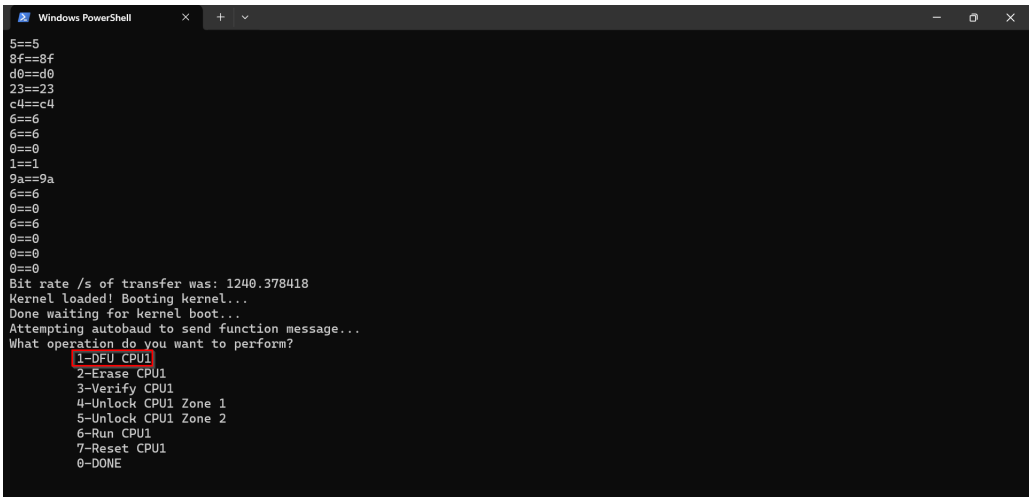

图 4-17. 命令串行闪存编程器下载应用程序

2. 命令执行后，需要启动应用程序。选择 *6-Run CPU1* 运行应用程序，并指定跳转地址 0x0008 0000（应用程序的闪存入口地址）。现在，已通过 SCI 引导加载到器件的应用程序会执行闪存中加载的应用程序。

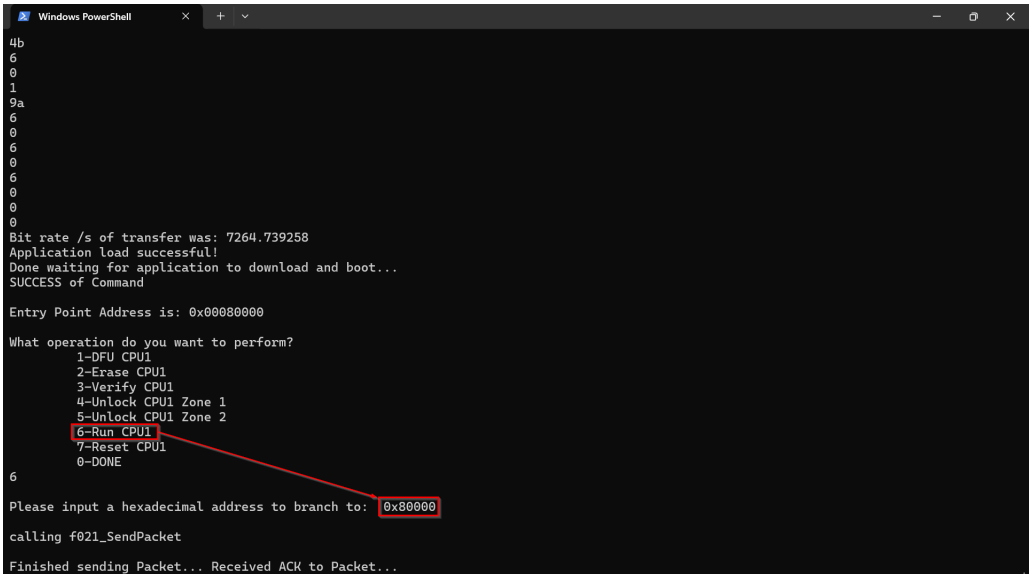

图 4-18. 将应用程序加载到闪存后运行

版权所有 © 2025 Texas Instruments Incorporated

4.2.3 CAN Boot

注意

虽然这些步骤是在 F280039C LaunchPad 上执行的，但这一通用流程也适用于支持自定义 BMSP 和引导定义表的其他 C2000 器件（表 2-3 中列出的所有器件）。有关目标器件的详细信息，请参阅相应的器件专用 TRM。

DCAN 闪存内核基于 ROM 引导加载程序，并与 C2000Ware 中提供的主机 PC 应用程序通信；该应用程序位于 C2000Ware (C2000Ware_x_xx_xx_xx > utilities > flash_programmers > dcan_flash_programmer)，会向主机反馈数据包接收情况以及所执行命令的完成状态。主机应用程序的源代码和可执行文件位于 dcan_flash_programmer 文件夹。有关内核功能的详细说明，请参阅 *CAN Flash Programming of C2000 Microcontrollers application note* [16]。

以下器件由 C2000 Ware 中的 dcan_flash_programmer 支持 ：

1. F28003x 2.
F280015x 3. F
28P65x

注意

F280015x LaunchPad 上的内置 CAN 收发器和连接器无法对应到任何可用的 CAN GPIO 分配，因此需要焊接跳线，将正确的 GPIO 与收发器连接 [32]。

注意

本文中的 DCAN 指 DCAN 闪存内核、DCAN 闪存编程器，以及博世设计的 D 版控制器局域网通信接口（CAN）[13]。本文所述 DCAN 闪存编程器对应的是 CAN 模块。

DCAN 闪存内核基于 DCAN ROM 加载器源代码开发。要使其能够擦除和编程闪存，必须集成 Flash API；具体通过链接 Flash API 实现。

在接收任何应用程序数据之前，F28P65x 和 F280015x DCAN 闪存内核会先擦除器件闪存，为编程做好准备。F28P65x 和 F280015x DCAN 闪存内核项目允许用户指定在写入应用程序之前要擦除哪些闪存区和扇区。更多详细信息请参阅应用说明《CAN Flash Programming of C2000 Microcontrollers》[16]。

擦除闪存中的相应位置后，即可开始加载应用程序。缓冲区用于存放连续接收到的应用程序代码块。当缓冲区写满，或检测到新的非连续数据
块时，内核会将缓冲区中的代码写入闪存。此过程会持续进行，直至接收完整个应用程序。

首次写入某个扇区之前，F28003x DCAN 闪存内核会检查该扇区是否已擦除。如果尚未擦除，则由 F28003x 闪存内核调用 Flash API 执行擦除操作。随后，内核将待写入内容填入缓冲区，并通过 Flash API 发出编程命令。写入完成后，闪存内核会调用 Flash API 验证该数据段是否已写入闪存的正确地址。所有内容复制到闪存后，项目会跳转到映像的入口地址。

闪存内核初始化 DCAN 模块后，该模块会等待主机发送固件映像。闪存内核每次从主机接收 8 字节，并将内容放入中间 RAM 缓冲区。随后，缓冲区内容会以 128 位或 512 位为单位写入闪存。

- F28P65x 和 F280015x DCAN 闪存内核项目支持 512 位编程 – F28P65x 器件也提供可选的 128 位编程项目 – F280015x 闪存 API 支持 128 位编程，但该闪存内核采用 512 位编程实现

- F28003x DCAN 闪存内核项目支持 128 位编程

版权所有 © 2025 Texas Instruments Incorporated

闪存中固件映像的所有段都会按照每次编程的位数进行对齐。

- 如果每次编程 128 位 (F28003x)，则应用程序的闪存段必须按 128 位边界对齐。在固件映像的链接器命令文件中，必须将所有已初始化段映射到闪存扇区，并在每项映射后添加 ALIGN(8) 指令，以确保满足 128 位对齐要求。
- 如果每次编程 512 位 (F280015x and F28P65x)，则应用程序的闪存段必须按 512 位边界对齐。在固件映像的链接器命令文件中，所有已初始化段
  都必须映射到闪存扇区，并在每项映射后添加 ALIGN(32) 指令，以确保满足 512 位对齐要求。

注意

ALIGN(x) 指令会插入填充字节，直到编程位置对齐到 *x* 字边界。C2000 的字长为 16 位，因此，16 位编程需要 ALIGN(1)，32 位编程需要 ALIGN(2)，以此类推。

C2000Ware 在相应器件的 examples 目录中提供了 CCS 闪存内核源文件和项目。这些项目包含一个构建后步骤，可将编译和链接生成的 .out 文件转换为 DCAN ROM 引导加载程序所需的引导十六进制格式，并以项目名称加 .txt 扩展名保存。

可查看 C2000Ware 中 *CAN Transceiver and Connector* 对应的 LaunchPad 或 controlCARD 原理图（C2000Ware_x_xx_xx > boards > (LaunchPads or controlCARDs) > DEVICE_NAME > Rev# > documentation），确认 LaunchPad 上 CAN 路由所需的 GPIO 分配。对于 LAUNCHXL-F280039C，收发器 RXD 内部连接到 GPIO5，TXD 内部连接到 GPIO4，因此必须配置引导选项 0x02。

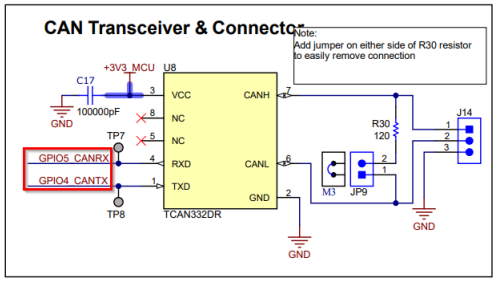

图 4-19. F280039C LaunchPad CAN 收发器和连接器原理图

1. 在 C2000Ware 中找到目标器件的 DCAN 闪存内核项目，并将其导入 CCS。例如，F28003x 的 DCAN 闪存内核位于 C2000Ware_x_x_xx_xx > driverlib > f28003x > examples > flash。2. 确认 DCAN 闪存内核项目的活动构建目标配置设为 *RAM*，因为内核必须链接为在 RAM 中执行。3. 如果使用 LaunchPad，请将预定义符号 *_LAUNCHXL_F280039C* 配置到 Project Properties > CCS Build > C2000 Compiler > Predefined Symbols 中，以便为 LaunchPad 使用正确的 GPIO 分配

（device.h）。

版权所有 © 2025 Texas Instruments Incorporated

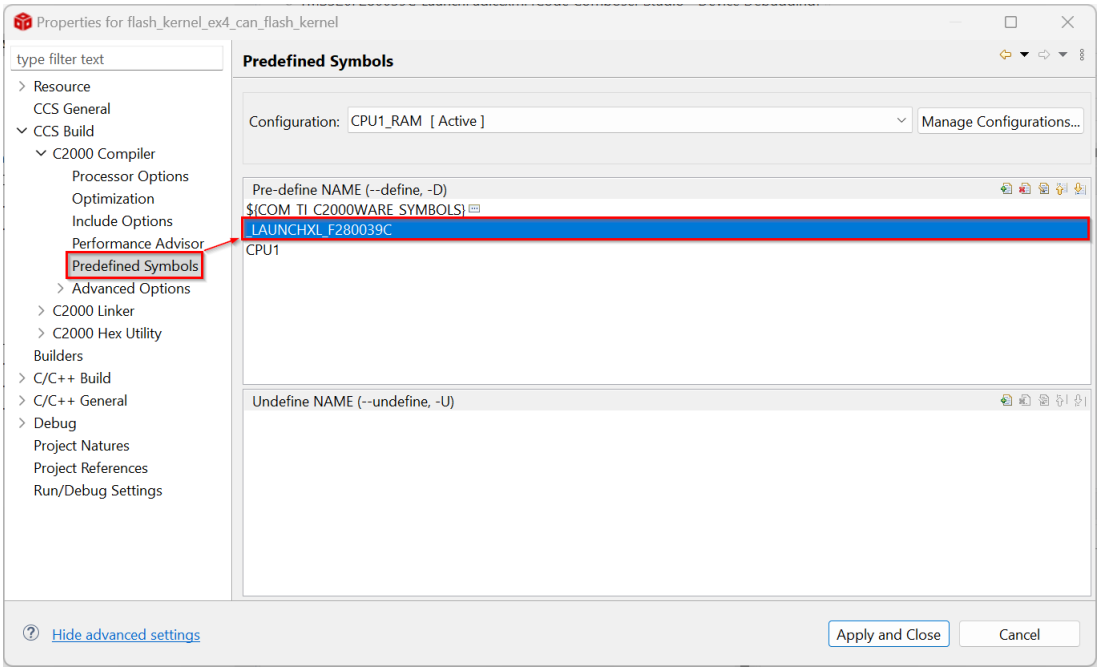

图 4-20. 添加预定义符号以设置正确的 LaunchPad CAN GPIO 分配 4. 上电时，器件 Boot ROM 使用片上 10MHz 振荡器（INTOSC2）提供时钟。必须在闪存内核中将此值设为主内部时钟源；复位时默认也使用该时钟。在 *device.h* 中，取消注释并使用第 295 行的 *#define USE_PLL_SRC_INTOSC*。注释掉 *#define USE_PLL_SRC_XTAL*。

5. 在 *DCAN_boot.c* 中，确认第 64 行包含 *bootloader_can_timing.h* 的本地副本，而不是 *include* 文件夹中的自动生成头文件。

```
54 //
55 // Included Files
56 //
57 #include "can.h"
58 #include "cpu1bootrom.h"
59 #include "cpu1brom_utils.h"
60 #include "inc/hw_memmap.h"
61 #include "inc/hw_sysctl.h"
62 #include "inc/hw_can.h"
63 #include "inc/hw_gpio.h"
64 #include "bootloader_can_timing.h"
65
66 #include "F021_F28003x_C28x.h"
67 #include "flash_programming_f28003x.h"
```

图 4-21. 包含本地 CAN 时序头文件

6. 在 *bootloader_can_timing.h* 中，确认第 51-53 行定义了以下 CAN 时序设置：

a. 1M bps bitrate
b. 20 Mhz CAN clock
c. 20 ms bit time

```
48 //
49 // CAN bit timing settings for running CAN at 100kbps with a 20MHz crystal
50 //
51 #define CAN_CALC_BITRATE   1000000U
52 #define CAN_CALC_CANCLK    20000000U
53 #define CAN_CALC_BITTIME   20U
```

图 4-22. 确认 CAN 时序设置

7. 构建内核项目。这些项目包含一个构建后步骤，可将编译和链接生成的 .out 文件转换为 DCAN ROM 引导加载程序所需的引导十六进制格式，并以示例名称加 txt 扩展名保存。

版权所有 © 2025 Texas Instruments Incorporated

a. 如果未生成 txt 输出文件，请按照第 4.1 节中的步骤检查是否已定义正确的构建后步骤来生成十六进制文件。

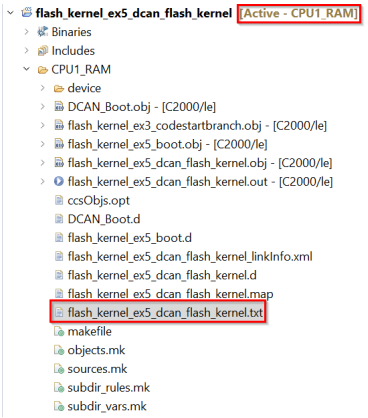

图 4-23. 生成 DCAN 内核 txt 输出文件

8. 在文本编辑器中打开闪存内核 txt 文件，并将第 3-4 字节改为 ***C0 7A***。这些字节包含用于配置 CAN 总线比特率的元数据。9. 对将由内核加载到闪存中的固件代码重复构建流程。a. 确认活动构建目标配置设为 ***Flash***。b. 确认已定义用于生成十六进制文件的正确构建后步骤。c. 构建固件项目。

在 CCS 中构建内核和固件项目后，应正确设置器件硬件，以便与运行 C2000Ware 所提供 dcan_flash_programmer 的主机 PC 通信。首先要正确配置引导模式选择引脚，使器件进入 CAN 引导模式。如果需要从外部主机将代码加载到片上闪存，可在支持的情况下使用默认 BMSP 配置 CAN 引导，或配置 BOOTPIN-CONFIG 和 BOOTDEF 寄存器。

用于启用 CAN 引导的默认 BMSP 见 TMS320F28003x Real-Time Microcontrollers data sheet。如果将 GPIO24 设为 1、GPIO32 设为 0，Boot ROM 就会跳转到 CAN 引导加载程序，并将 CANRXA 连接到 GPIO5、将 CANTXA 连接到 GPIO4，无需编程器件寄存器。

但是，如果用户希望使用不同 GPIO 分配方式灵活配置 CAN 引导，则必须针对相应引导选项配置 OTP 或仿真 BOOTCONFIG 和 BOOTDEF 寄存器。请参阅 TMS320F28003x Real-Time Microcontrollers 数据手册中的 ***GPIO Assignments***，了解符合 GPIO 要求的 CAN 引导选项。

运行示例所需的硬件包括一台连接到 CAN 收发器的 C2000 器件，以及一台 PEAK PCAN-USB Pro FD 分析仪。

1. LaunchPad 器件内置 CAN 收发器。确认 PEAK PCAN-USB Pro FD 分析仪通过接地线、CAN-Lo 和 CAN-Hi 连接到 LaunchPad。2. 确认板载 CAN 路由开关已拨至低电平（XCVR 位置），使收发器能够通过 GPIO 通信。

版权所有 © 2025 Texas Instruments Incorporated

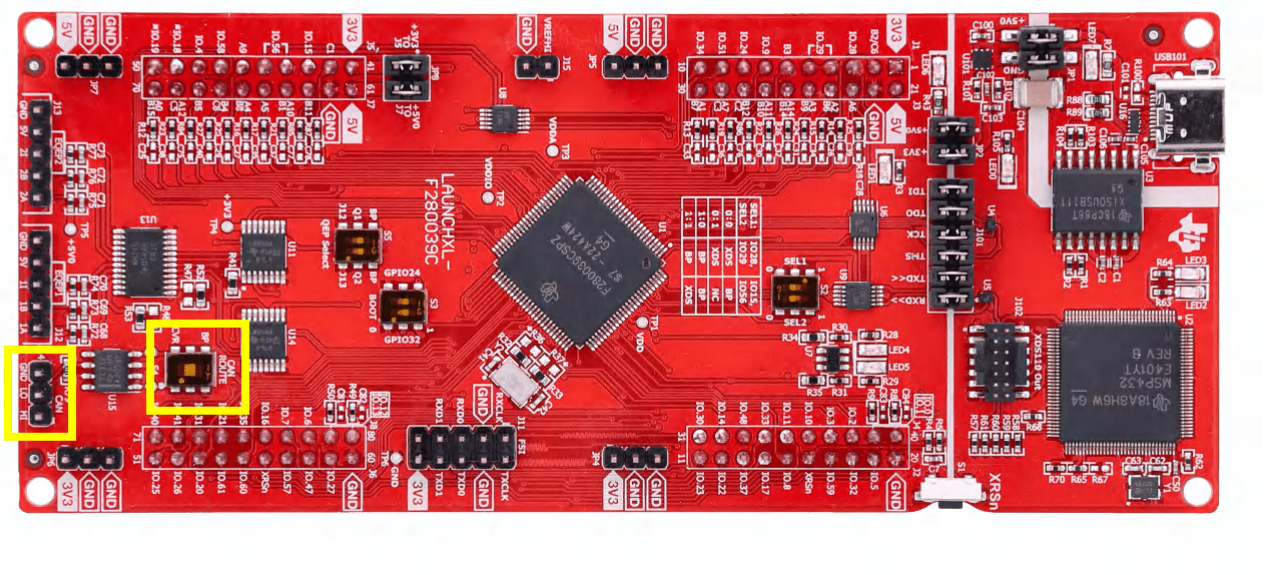

图 4-24. 设置 CAN 路由开关

Note

For controlCards, a custom designed CAN transceiver board and the HSEC-180-pin controlCard Docking Station needs to be used. The custom-designed transceiver board is connected to the controlCard using four connections: ground, 3.3V, CANTX and CANRX.

现在需要将器件配置为模拟 CAN 引导，使用引导选项 0x02，并将 CANRXA = 连接到 GPIO5、CANTXA = 连接到 GPIO4。发送第一帧 CAN 数据的超时时间只有 ≈10 秒。

1. 在 CCS 中打开一个工作区。2. 选择 Vi
ew > Target Configurations。

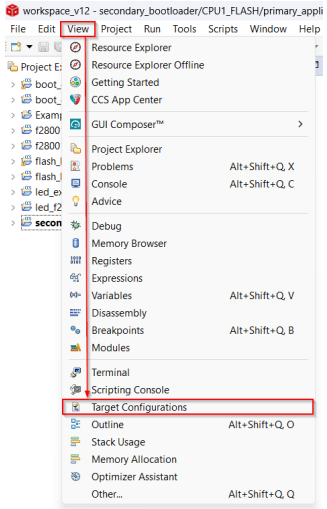

图 4-25. 在 CCS 中打开目标配置菜单

3. 可将该器件的项目导入 CCS 并使用该项目连接器件；也可从 C2000Ware 复制目标配置文件（.ccxml），路径为 (C2000Ware_x_xx_xx_xx > device_support > DEVICE_FAMILY > common > targetConfigs)，然后将其复制到 *User Defined* target configurations。a. 找到器件的目标配置文件，然后右键单击并手动启动。

版权所有 © 2025 Texas Instruments Incorporated

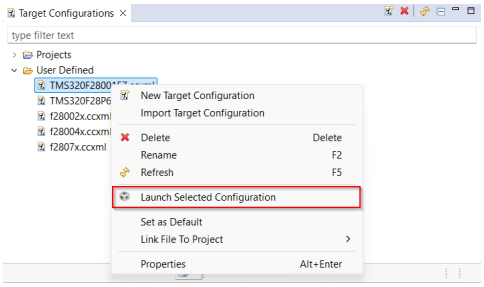

图 4-26. 在 CCS 中启动目标配置

4. CCS 打开调试窗口后，选择目标 CPU 并连接到目标器件。

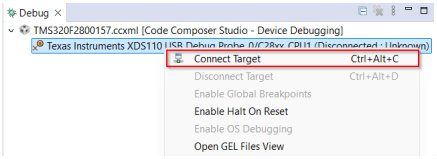

图 4-27. 连接到目标内核

5. 如果弹出窗口提示 Boot ROM 中存在断点但没有可用的调试信息，或断点位于程序代码之外，请按照第 5.3 节调试 Boot ROM。6. 加载符号后，选择 View > Memory Browser 打开存储器浏览器。

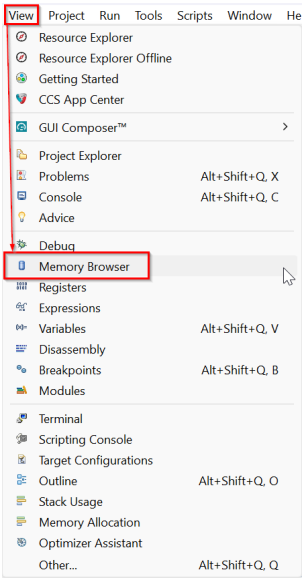

图 4-28. 转到存储器浏览器

7. 在存储器浏览器选项卡中，转到地址 0xD00。请注意，地址 0xD00 用于指定带有效性密钥（EMU-BOOTPIN-CONFIG）的 BMSP；地址 0xD04-0xD05 用于指定引导定义（EMU-BOOTDEF-LOW）。

8. 目标是配置零引脚 CAN 引导，使 CANRXA = 连接到 GPIO5、CANTXA = 连接到 GPIO4，因此必须禁用所有 BMSP，并将 EMU-BOOTDEF-LOW 的最低索引设置为 0x02。如果将引导选项写入 EMU-BOOTDEF-LOW 中的其他条目，则不会选择预期的引导模式。

。

a. 将 0xD00-0xD01（EMU-BOOTPIN-CONFIG）设为 0x5AFF FFFF

。b. 将 0xD04（EMU-BOOTDEF-LOW）设为 0x0002。

注意

零引脚引导是指器件自动引导到 BOOTDEF 表中定义的第一项。为此需禁用所有 BMSP，使器件仅考虑一个引导选项。

版权所有 © 2025 Texas Instruments Incorporated

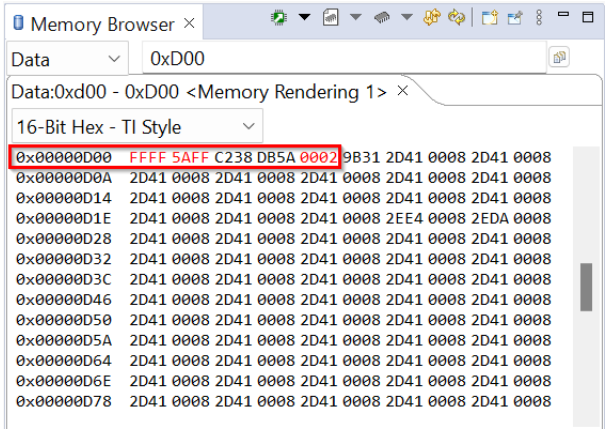

图 4-29. 模拟零引脚 CAN 引导：CANRXA=连接到 GPIO5，CANTXA=连接到 GPIO4

9. 复位 CPU 并执行外部复位（XRSn），然后单击 ***Resume*** 开始引导序列。10. 如果 Boot ROM 中出现断点，但没有可用的调试信息，或断点位于程序代码之外，请按照第 5.3 节加载 Boot ROM 符号。随后确认器件处于 CAN 引导模式。

随后，ROM 中的 CAN 引导加载程序（引脚分配由引导选项 0x02 指定）开始执行，并等待主机发送 CAN 帧。此时，器件已准备好从主机接收
代码。

注意

CAN 引导加载程序接收 CAN 帧的时间窗口为 10 秒。如果器件引导选项提供 SEND_TEST 模式，则该模式可取消超时。这些选项使用与对应引导选项相同的 GPIO 引脚进行通信，但会在引导加载前先通过 GPIO 发送一帧，以便确认模块工作正常。对于 F280039x 器件，可使用引导选项 0x82，在沿用引导选项 0x02 的相同 GPIO 分配方式时避免超时。

命令行 PC 实用程序是一种便于集成到生产线编程等脚本环境中的编程方案。该程序使用 Microsoft Visual Studio 和 C++ 编写。项目和源代码位于 C2000Ware (C2000Ware_x_xx_xx_xx > utilities > flash_programmers > dcan_flash_programmer)。

主机负责向 MCU 发送 DCAN 内核映像和闪存（固件）映像。此处使用 PEAK PCAN-USB Pro FD CAN 总线分析仪作为主机。闪存编程器项目使用 Visual Studio 2019 构建和运行。主机编程器使用 PEAK 提供的 PCAN_Basic API [17]，可通过该 API 在 CAN 分析仪上发送和接收 CAN 帧。

注意：PEAK PCAN-USB Pro FD CAN 总线分析仪可向后兼容，既能接收 CAN 帧，也能接收 CAN-FD 帧。

对于 F28003x 器件，Boot ROM 会将 CAN 模块的时钟切换为外部时钟源。LaunchPad 和 controlCARD 上的外部时钟频率为 20MHz。Boot ROM 将标称比特率配置为 100Kbps。主机 CAN 编程器会将 PEAK CAN 分析仪配置为相同的时钟和标称比特率。

主机先初始化分析仪以便使用 CAN，再以每次 2 字节的方式发送内核，然后以每次 8 字节的方式发送映像，并在每帧之间延迟 10ms，以便 Flash API 有时间将接收的数据写入闪存。固件映像写入完成后，主机 CAN 编程器退出。

使用此工具编程 C2000 器件前，请确认目标板已复位、处于上述配置的 CAN 引导模式，并已连接到 PEAK PCAN-USB Pro FD CAN 总线分析仪。下面介绍该工具在单核 CAN 引导时的命令行用法，必需参数为 -d、-k 和 -a；

版权所有 © 2025 Texas Instruments Incorporated

这些参数均为必需项。可使用 -v 启用详细输出。有关工具参数的更多信息请参阅 [4]。

```
dcan_flash_programmer.exe –d DEVICE -k KERNEL_FILE -a APPLICATION_FILE -v
```

1. 打开包含已编译 dcan_flash_programmer 可执行文件的文件夹 (C2000Ware_x_xx_xx_xx > utilities > flash_programmers > dcan_flash_programmer)。2. 打开终端，运行 dcan_flash_programmer 可执行文件并使用以下命令：

```
dcan_flash_programmer.exe –d DEVICE_NAME –k <path_to_kernel_hex> -a <path_to_application_hex> -v
```

首先，工具使用引导加载程序将 DCAN 闪存内核加载到器件 RAM。终端中会显示通过 CAN 总线传输的字节。随后内核开始执行，并加载应用程序文件；如图 4-30 和图 4-31 所示，内核会使用命令行参数 -a 指定的文件编程闪存。若成功加载到闪存，内核就会跳转到应用程序并开始执行。

图 4-30. 已加载 DCAN 闪存编程器内核

图 4-31. 已加载 DCAN 闪存编程器应用程序

版权所有 © 2025 Texas Instruments Incorporated

4.2.4 CAN-FD Boot

注意

虽然这些步骤是在 F280039C LaunchPad 上执行的，但这一通用流程也适用于支持自定义 BMSP 和引导定义表的其他 C2000 器件（表 2-3 中列出的所有器件）。有关目标器件的详细信息，请参阅相应的器件专用 TRM。

ROM 引导加载程序只能将代码加载到 RAM，因此会用它将闪存内核加载到 RAM，以便随后将代码存储到闪存中；详情见第 3.2 节。CAN 闪存内核基于 ROM 引导加载程序，并与 C2000Ware 中提供的主机 PC 应用程序通信 (C2000Ware_x_xx_xx_xx > utilities > flash_programmers > can_flash_programmer)，同时向主机反馈数据包接收情况及所执行命令的完成状态。主机应用程序的源代码和可执行文件位于

can_flash_programmer 文件夹。有关内核功能的更多信息，请参阅应用说明《CAN Flash Programming of C2000 Microcontrollers》[16]。

以下器件由 C2000 Ware 中的 can_flash_programmer 支持：

1. F28003x 2. F28P55x 3. F28P65x

注意

本文中的 MCAN、MCAN 闪存内核、CAN 闪存编程器等术语，均指模块化控制器局域网（MCAN）。MCAN 与控制器局域网灵活数据速率（CAN-FD）[15] 是可互换的术语。本文所述 CAN 闪存编程器对应的是 MCAN 模块。

闪存内核项目以 MCAN ROM 引导加载程序为基础，其中直接调用经过修改、可用于写入闪存的 MCAN_Boot 函数。闪存内核对 MCAN 模块的初始化方式与引导加载程序相同。内核会根据引导模式，在初始化时设置 MCAN 模块时钟源、标称和数据比特率、GPIO 引脚等。

接收任何应用程序数据之前，F28P55x 内核会先擦除器件闪存，为编程做好准备。此外，F28P55x MCAN 闪存内核项目允许用户指定写入应用程序前要擦除哪些闪存区和扇区。更多详细信息请参阅应用说明《CAN Flash Programming of C2000 Microcontrollers》中的 *Custom Flash Bank and Sector Erase* 部分 [16]。

F28003x 内核会在编程应用程序前逐个检查闪存扇区是否已擦除。如果扇区尚未擦除，内核会先擦除该扇区，再写入应用程序数据。

F28P65x 内核会在应用程序下载开始时擦除闪存。闪存擦除可能需要几秒钟。请注意，即使应用程序加载过程显示失败，闪存仍可能正在擦除。

擦除闪存中的相应位置后，即可开始加载应用程序。闪存内核每次从主机接收 64 字节，并将内容放入中间 RAM 缓冲区。随后，缓冲区内容会以 128 位或 512 位为单位写入闪存。

- F28003x 和 F28P65x MCAN 闪存内核每次写入 128 位。
- F28P55x MCAN 闪存内核每次写入 512 位。

RAM 缓冲区填满待写入闪存的内容后，Flash API 会发出编程命令。写入完成后，闪存内核会调用 Flash API，验证该数据段是否已写入闪存的正确地址。所有内容复制到闪存后，项目会跳转到映像的入口地址。

固件映像中存储在闪存的所有段，都必须按照每次编程的位数进行对齐。

版权所有 © 2025 Texas Instruments Incorporated

- 如果每次编程 128 位 (F28003x and F28P65x)，则应用程序的闪存段必须按 128 位边界对齐。在固件映像的链接器命令文件中，必须将所有已初始化段映射到闪存扇区。每项映射后都要添加 ALIGN(8) 指令，以确保满足 128 位对齐要求。
- 如果每次编程 512 位 (F28P55x)，则应用程序的闪存段必须按 512 位边界对齐。在固件映像的链接器命令文件中，必须将所有已初始化段映射到闪存扇区。每项映射后都要添加 ALIGN(32) 指令，以确保满足 512 位对齐要求。

注意

ALIGN(x) 指令会插入填充字节，直至编程位置对齐到 ***x*** 字边界。C2000 的字长为 16 位，因此，16 位编程需要 ALIGN(1)，32 位编程需要 ALIGN(2)，以此类推。

用于传输应用程序数据的协议遵循 MCAN ROM 加载器协议。原始 MCAN ROM 加载器协议使用的标称比特率为 1Mbps；在标称位时序下，主机会向目标器件每帧发送 64 字节。该协议的数据比特率为 2Mbps，用于数据位时序。

C2000Ware 在相应器件的 examples 目录中提供了 CCS 闪存内核源文件和项目。这些项目包含一个构建后步骤，可将编译和链接生成的 .out 文件转换为 MCAN ROM 引导加载程序所需的引导十六进制格式，并以项目名称加 txt 扩展名保存。

可查看 C2000Ware 中 ***CAN Transceiver and Connector*** 对应的 LaunchPad 或 controlCARD 原理图（C2000Ware_x_xx_xx_xx > boards > (LaunchPads or controlCARDs) > DEVICE_NAME > Rev# > documentation），确认 LaunchPad 上 CAN 路由所需的 GPIO 分配。对于 LAUNCHXL-F280039C，收发器 RXD 内部连接到 GPIO5，TXD 内部连接到 GPIO4，因此必须配置 BOOTDEF 0x02。

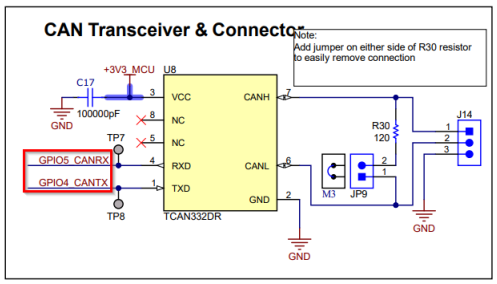

图 4-32. F280039C LaunchPad CAN 收发器和连接器原理图

1. 在 C2000Ware 中找到目标器件的 MCAN 闪存内核项目，并将其导入 CCS。例如，F28003x 的 MCAN 闪存内核位于 C2000Ware_x_x_xx_xx > driverlib > f28003x > examples > flash。2. 确认该项目的活动构建目标配置设为 ***RAM***，因为内核必须链接为在 RAM 中执行。3. 将预定义符号 ***_LAUNCHXL_F280039C*** 配置到 Project Properties > CCS Build > C2000 Compiler > Predefined Symbols 中，以便在 LaunchPad 的 device.h 中使用正确的 GPIO 分配。

版权所有 © 2025 Texas Instruments Incorporated

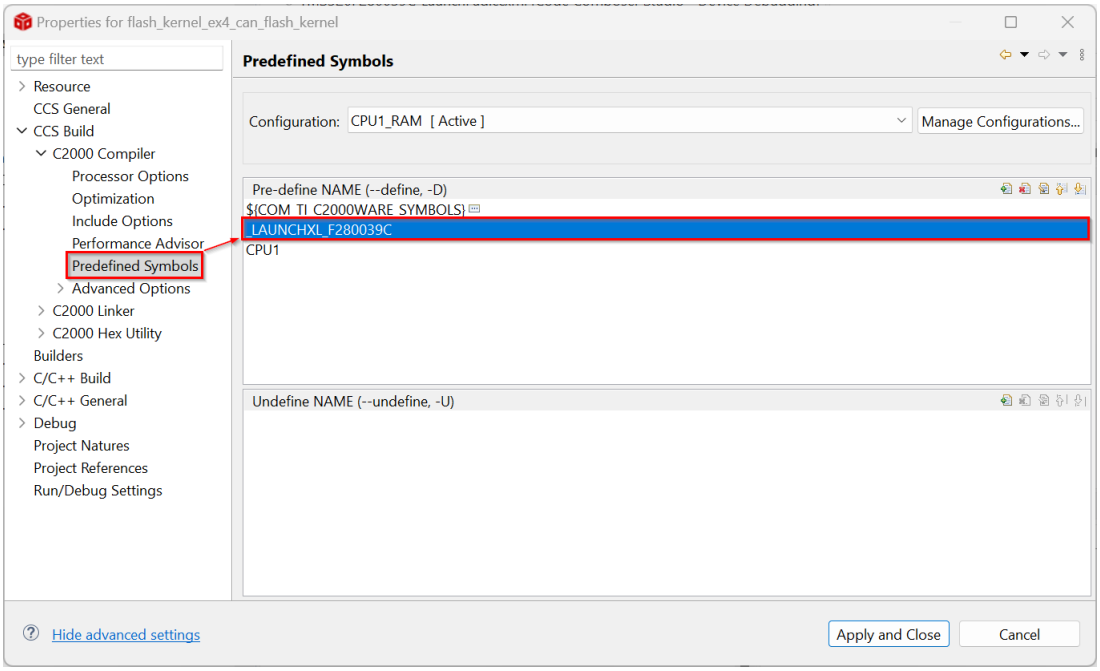

图 4-33. 添加预定义符号以设置正确的 LaunchPad CAN GPIO 分配

4. 上电时，器件 Boot ROM 使用片上 10MHz 振荡器（INTOSC2）提供时钟。必须将该时钟设为主内部时钟源；复位时默认也使用该时钟。在 device.h 中，取消注释并使用第 295 行的 ***#define USE_PLL_SRC_INTOSC***。注释掉 ***#define USE_PLL_SRC_XTAL***。

5. 构建内核项目。这些项目包含一个构建后步骤，可将编译和链接生成的 .out 文件转换为 MCAN ROM 引导加载程序所需的引导十六进制格式，并以示例名称加 txt 扩展名保存。

   a. 如果未生成 txt 输出文件，请按照第 4.1 节中的步骤检查是否已定义正确的构建后步骤来生成十六进制文件。

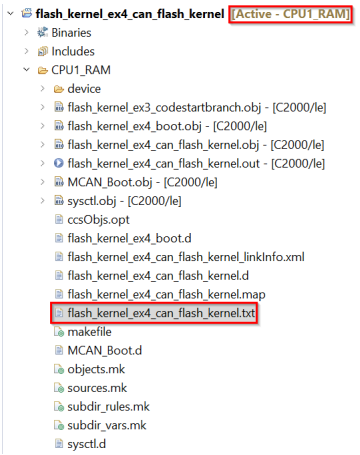

图 4-34. 生成 CAN 内核 txt 输出文件

6. 对将由内核加载到闪存中的应用程序代码重复构建流程。a. 确认活动构建目标配置设为 ***Flash***。b. 确认已定义用于生成十六进制文件的正确构建后步骤。c. 构建固件项目。

在 CCS 中构建内核和固件项目后，应正确设置器件硬件，以便与运行 C2000Ware 所提供 can_flash_programmer 的主机 PC 通信。首先要正确配置引导模式选择引脚，使器件进入 CAN-FD 引导模式。

版权所有 © 2025 Texas Instruments Incorporated

如果用户需要从外部主机将代码加载到片上闪存，则必须配置 BOOTPIN-CONFIG 和 BOOTDEF 寄存器，因为默认引导选项不支持此模式。请参阅 TMS320F28003x Real-Time Microcontrollers 数据手册中的 GPIO 分配说明，了解符合 GPIO 要求的 CAN-FD 引导选项。

运行示例所需的硬件包括一台连接到 CAN 收发器的 C2000 器件，以及一台 PEAK PCAN-USB Pro FD 分析仪。

1. LaunchPad 器件内置 CAN 收发器。确认 PEAK PCAN-USB Pro FD 分析仪通过接地线、CAN-Lo 和 CAN-Hi 连接到 LaunchPad。2. 确认板载 CAN 路由开关已拨至低电平（XCVR 位置），使收发器能够通过 GPIO 通信。

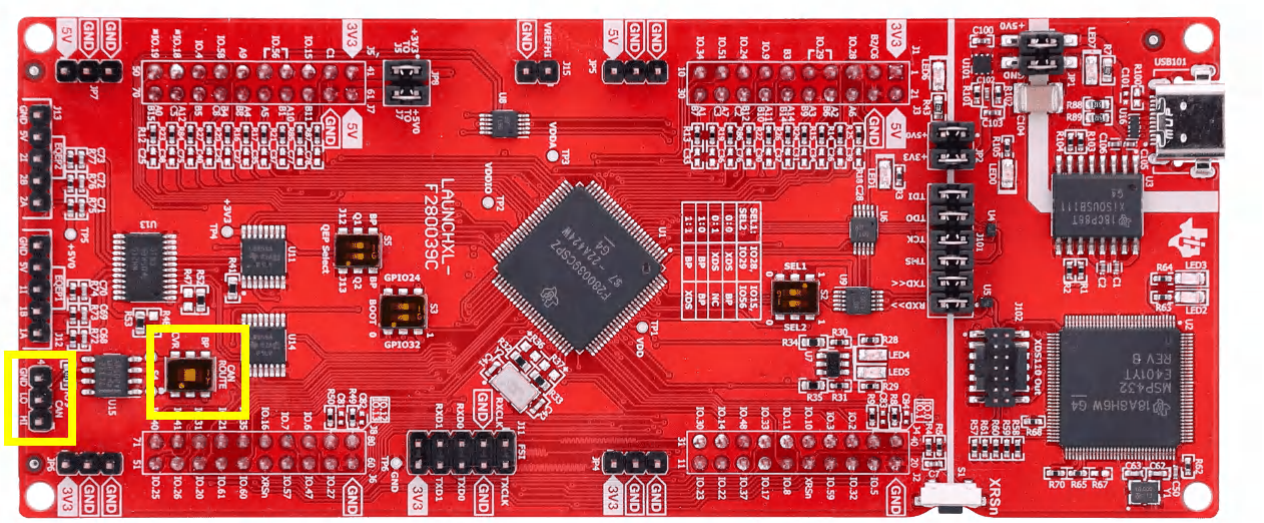

图 4-35. 设置 CAN 路由开关

注意

对于 controlCARD，需要使用定制 CAN 收发器板以及 HSEC-180-pin ControlCARD 底座。定制收发器板通过以下四条连接线接到 ControlCARD：地线、3.3V、CANTX 和 CANRX。

需要将器件配置为模拟零引脚 CAN 引导，使用引导选项 0x02，并将 CANRXA = 连接到 GPIO5、CANTXA = 连接到 GPIO4。随后还需将器件配置为模拟零引脚 CAN-FD 引导，使用引导选项 0x08，并将 CANRXA = 连接到 GPIO5、CANTXA = 连接到 GPIO4。请注意，发送第一帧 CAN 数据的时间约为 10 秒。

1. 在 CCS 中打开一个工作区。2. 选择 View > Target Configurations。

版权所有 © 2025 Texas Instruments Incorporated

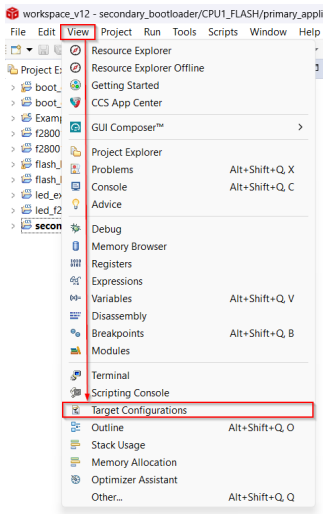

图 4-36. 在 CCS 中打开目标配置菜单

3. 可将该器件的项目导入 CCS，并使用该项目连接器件；也可从 C2000Ware 复制目标配置文件（.ccxml），路径为 (C2000Ware_x_xx_xx_xx > device_support > DEVICE_FAMILY > common > targetConfigs)，然后将其复制到 *User Defined* target configurations。

a. 找到器件的目标配置文件，然后右键单击并手动启动：

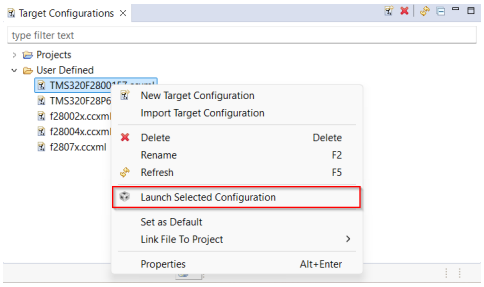

图 4-37. 在 CC S 中启动目标配置

4. CCS 打开调试窗口后，选择目标 CPU 并连接到目标器件。

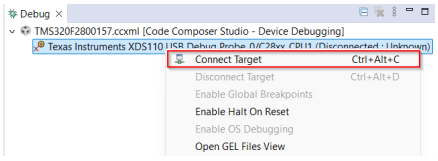

图 4-38. 连接到目标内核

5. 如果弹出窗口提示 Boot ROM 中存在断点但没有可用的调试信息，或断点位于程序代码之外，请按照第 5.3 节调试 Boot ROM。6. 加载符号后，选择 View > Memory Browser 打开存储器浏览器。

版权所有 © 2025 Texas Instruments Incorporated

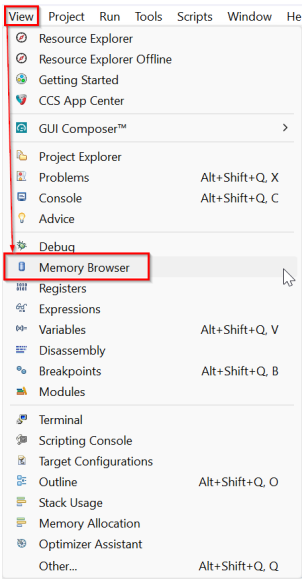

图 4-39. 转到存储器浏览器

7. 在存储器浏览器选项卡中，转到地址 0xD00。请注意，地址 0xD00 用于指定带有效性密钥（EMU-BOOTPIN-CONFIG）的 BMSP；地址 0xD04-x0D05 用于指定引导定义（EMU-BOOTDEF-LOW）。

8. 目标是配置零引脚 CAN 引导，使 CANRXA = 连接到 GPIO5、CANTXA = 连接到 GPIO4，因此必须禁用所有 BMSP，并将 EMU-BOOTDEF-LOW 的最低索引设置为 0x08。如果将引导选项写入 EMU-BOOTDEF-LOW 中的其他条目，则不会选择预期的引导模式。

。

a. 将 0xD00-0xD01（EMU-BOOTPIN-CONFIG）设为 0x5AFF FFFF

。b. 将 0xD04（EMU-BOOTDEF-LOW）设为 0x0008。

注意

在零引脚引导模式下，除 EMU-BOOTDEF-LOW 的低八字节外，EMU-BOOTDEF-LOW 和 EMU-BOOTDEF-HIGH 均可设为任意值，因为这些值会被忽略。

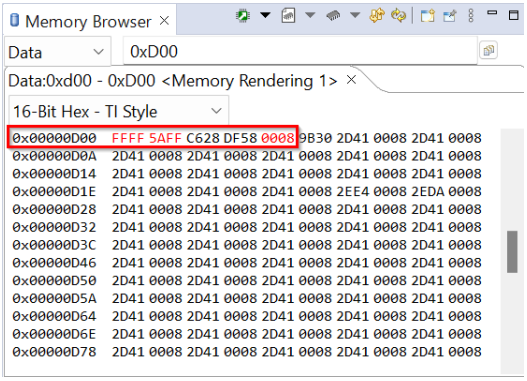

图 4-40. 模拟零引脚 MCAN 引导，CANRXA=连接到 GPIO5，CANTXA=连接到 GPIO4

---

9. 复位 CPU 并执行外部复位（XRSn），然后单击 ***Resume*** 开始引导序列。10. 如果 Boot ROM 中出现断点，但没有可用的调试信息，或断点位于程序代码之外，请按照第 5.3 节加载 Boot ROM 符号。随后确认器件处于 CAN 引导模式。

随后，ROM 中的 MCAN 引导加载程序（引脚分配由引导选项 0x08 指定）开始执行，并等待主机发送 CAN 帧。此时，器件已准备好从主机接收代码。

版权所有 © 2025 Texas Instruments Incorporated

注意

MCAN 引导加载程序接收 CAN-FD 帧的时间窗口为 10 秒。如果器件引导选项提供 SEND_TEST 模式，则该模式可取消超时。这些选项使用与对应引导选项相同的 GPIO 引脚进行通信，但会在引导加载前先通过 GPIO 发送一帧，以便确认模块工作正常。对于 F280039x 器件，可使用引导选项 0x68，在沿用引导选项 0x08 的相同 GPIO 分配方式时避免超时。

命令行 PC 实用程序是一种便于集成到生产线编程等脚本环境中的编程方案。该程序使用 Microsoft Visual Studio 和 C++ 编写。项目和源代码位于 C2000Ware (C2000Ware_x_xx_xx_xx > utilities > flash_programmers > can_flash_programmer)。

主机负责向 MCU 发送 MCAN 内核映像和闪存（固件）映像。此处使用 PEAK PCAN-USB Pro FD CAN 总线分析仪作为主机。闪存编程器项目使用 Visual Studio 2019 构建和运行。主机编程器使用 PEAK 提供的 PCAN_Basic API [17]，可通过该 API 在 CAN 分析仪上发送和接收 CAN-FD 帧。

注意

PEAK PCAN-USB Pro FD CAN 总线分析仪可向后兼容，既能接收 CAN 帧，也能接收 CAN-FD 帧。

对于 F28003x 器件，Boot ROM 会将 MCAN 模块的时钟切换为外部时钟源。LaunchPad 和 ControlCARD 上的外部时钟频率为 20MHz。Boot ROM 将标称比特率配置为 1Mbps、数据比特率配置为 2Mbps。主机 CAN 编程器会将 PEAK CAN 分析仪配置为相同的时钟、标称比特率和数据比特率。

主机初始化分析仪以便使用 CAN-FD，然后以每次 64 字节的方式发送内核，再以每次 64 字节的方式发送映像，并在每帧之间延迟 100ms，以便 Flash API 有时间将接收的数据写入闪存。固件映像写入完成后，主机 CAN 编程器退出。

使用此工具编程 C2000 器件前，请确认目标板已复位、处于上述配置的 CAN 引导模式，并已连接到 PEAK PCAN-USB Pro FD CAN 总线分析仪。下面介绍该工具在单核 MCAN 引导时的命令行用法，必需参数为 -d、-k 和 -a。可使用 -v 启用详细输出。有关工具参数的更多信息，请参阅 [14]。

```
can_flash_programmer.exe –d DEVICE -k KERNEL_FILE -a APPLICATION_FILE -v
```

1. 打开包含已编译 dcan_flash_programmer 可执行文件的文件夹 (C2000Ware_x_xx_xx_xx > utilities > flash_programmers > can_flash_programmer)。2. 打开终端，运行 can_flash_programmer.exe，并使用以下命令：

```
can_flash_programmer.exe –d DEVICE_NAME –k <path_to_kernel_hex> -a <path_to_application_hex> -v
```

该工具使用引导加载程序将 MCAN 闪存内核加载到器件 RAM。终端中会显示通过 MCAN 总线传输的字节。随后内核开始执行，并加载由命令行参数 -a 指定的文件以编程闪存，如图 4-41 和图 4-42 所示。若成功加载到闪存，内核就会跳转到应用程序并开始执行。

版权所有 © 2025 Texas Instruments Incorporated

图 4-41. 已加载 MCAN 闪存编程器内核

图 4-42. 已加载 MCAN 闪存编程器应用程序

4.2.5 USB Boot

> 注意：C2000 LaunchPad 没有支持 USB 数据传输的 USB 外设，而 controlCARD 具备该功能，因此以下步骤基于 F28379D controlCARD Rev 1.3。更重要的是，早期芯片版本存在已知缺陷，导致 USB 引导加载程序无法与主机 PC 通信，因此 F28379D 至少必须使用 Rev C 芯片版本。

USB 闪存内核基于 ROM 引导加载程序，与 C2000Ware 中提供的主机 PC 应用程序通信 (C2000Ware_x_xx_xx_xx > utilities > flash_programmers > usb_flash_programmer)，并执行与 ROM 引导加载程序基本相同的操作。由于 ROM 引导加载程序配备了 Flash API，因此能够擦除和编程闪存，以完成固件更新。

版权所有 © 2025 Texas Instruments Incorporated

注意

本节介绍 CPU1 USB 引导加载。有关 USB 内核功能和 CPU2 用法的详细说明，请参阅 *USB Flash Programming of C2000 Microcontrollers application report* [20]。

C2000Ware 在相应器件的 examples 目录中提供了 CCS 闪存内核源文件和项目。这些项目包含一个构建后步骤，可将编译和链接生成的 .out 文件转换为 SCI ROM 引导加载程序所需的引导十六进制格式，并以项目名称加 .dat 扩展名保存。

1. 在 C2000Ware 中找到目标器件对应的 USB 闪存内核项目，并将其导入 CCS。a. 对于 F2837xD 器件，内核位于 C2000Ware_x_xx_xx_ > device_support > f2837xD > dual > F2837xD_usb_flash_kernels。
2. 确认 USB 闪存内核项目的活动构建目标配置设为 ***RAM***，因为内核必须链接为在 RAM 中执行。

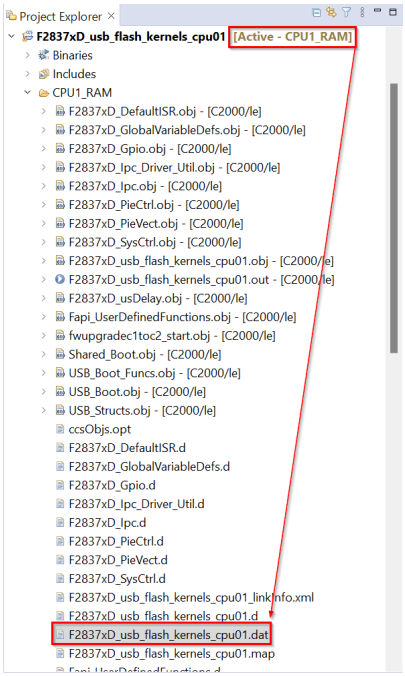

图 4-43. 生成正确的 USB 内核 Bin 输出

3. 构建内核项目。这些项目包含一个构建后步骤，可将编译和链接生成的 .out 文件转换为 USB ROM 引导加载程序所需的正确二进制格式，并以示例名称加 .dat 扩展名保存。

注意：如果未生成 .dat 输出文件，请按照第 4.1 节中的步骤检查是否已定义正确的构建后步骤来生成二进制文件。

4. 对将由内核加载到闪存中的固件代码重复构建流程。a. 确认活动构建目标配置设为 Flash。b. 按照第 4.1 节中的步骤检查是否已定义用于生成二进制文件的正确构建后步骤。c. 构建固件项目。

在 CCS 中构建内核后，应正确设置器件，以便与主机 PC 上运行的 usb_flash_programmer 通信。

1. 将 controlCARD 上的 mini-USB 和 micro-USB 连接到主机 PC。

版权所有 © 2025 Texas Instruments Incorporated

2. 设置 controlCARD GPIO，使其可通过板载 XDS100v2 仿真器在 CCS 中进行调试 [20]。有关连接选项，请参阅应用报告《USB Flash Programming of C2000™ Microcontrollers》中的 *ROM Bootloader* 部分。

a. 将 A:SW1 的位置 1 拨到 On（向上），将位置 2 拨到 Off（向下）。

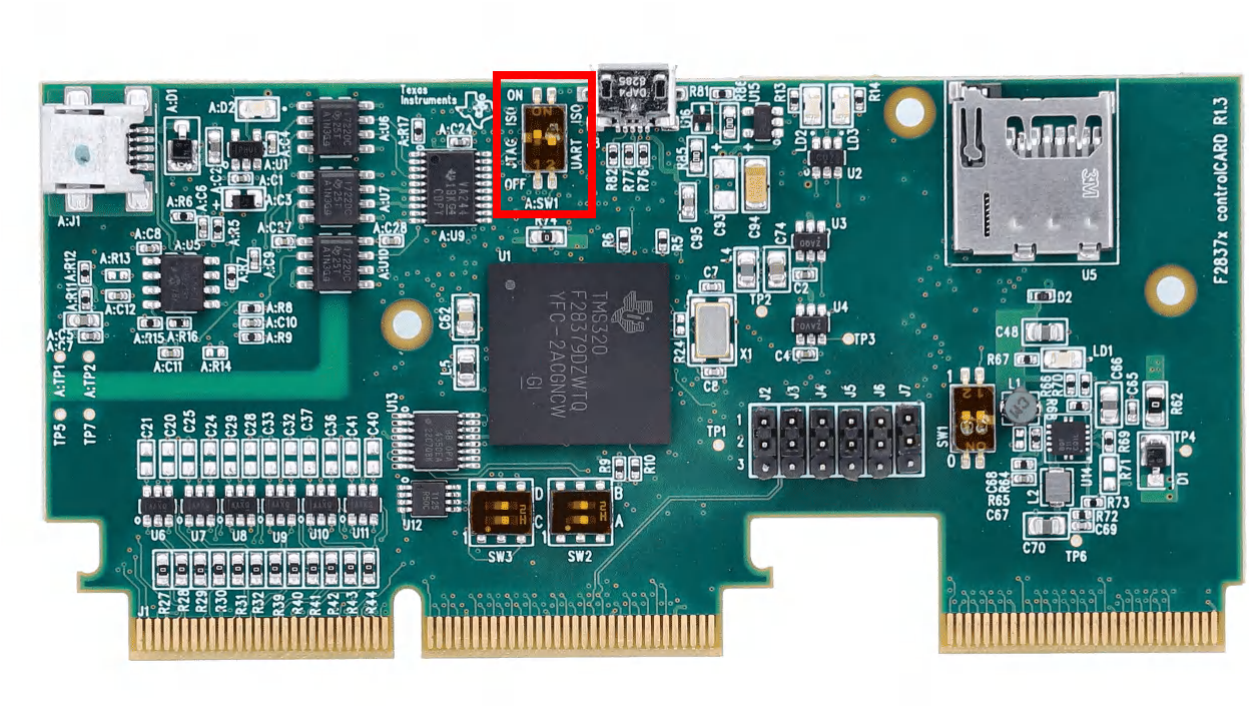

图 4-44. 配置 F2837xD controlCARD，以便通过 CCS 和板载 XDS 仿真器进行调试 3. 在 CCS 中打开一个工作区。4. 选择 View > Target Configurations。

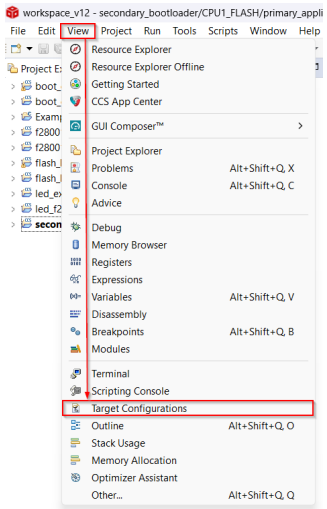

图 4-45. 在 CCS 中打开目标配置菜单

5. 可将该器件的项目导入 CCS，并使用该项目连接器件；也可从 C2000Ware 复制目标配置文件（.ccxml），路径为 (C2000Ware_x_xx_xx_xx > device_support > DEVICE_FAMILY > common > targetConfigs)，然后将其复制到 *User Defined* target configurations。a. 找到器件的目标配置文件，然后右键单击并手动启动。

版权所有 © 2025 Texas Instruments Incorporated

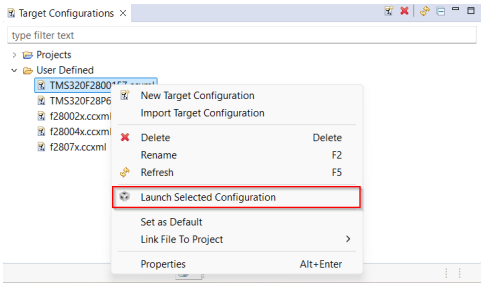

图 4-46. 在 CCS 中启动目标配置

6. CCS 打开调试窗口后，选择目标 CPU 并连接到目标器件。

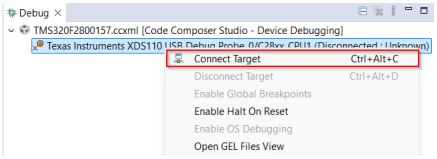

图 4-47. 连接到目标内核

7. 如果弹出窗口提示 Boot ROM 中存在断点但没有可用的调试信息，或断点位于程序代码之外，请按照第 5.3 节调试 Boot ROM。8. 加载符号后，选择 View > Memory Browser 打开存储器浏览器。

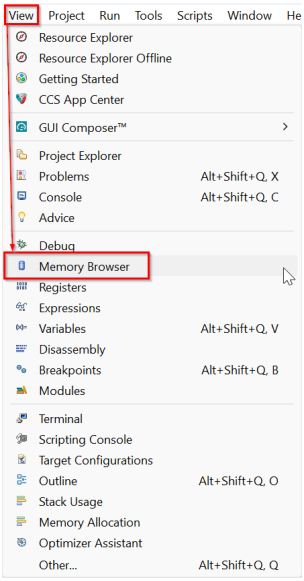

图 4-48. 转到存储器浏览器

9. 在存储器浏览器选项卡中，转到地址 0xD00。10. 将器件配置为仿真 ***USB Boot***。在 TMS320F28003xD Real-Time Microcontrollers 技术参考手册的 ***Configuring Get Boot Options*** 部分中，BMODE 的值定义为 0x0C。将地址 0xD00 编程为 0x0C5A。请参阅 [23]。

版权所有 © 2025 Texas Instruments Incorporated

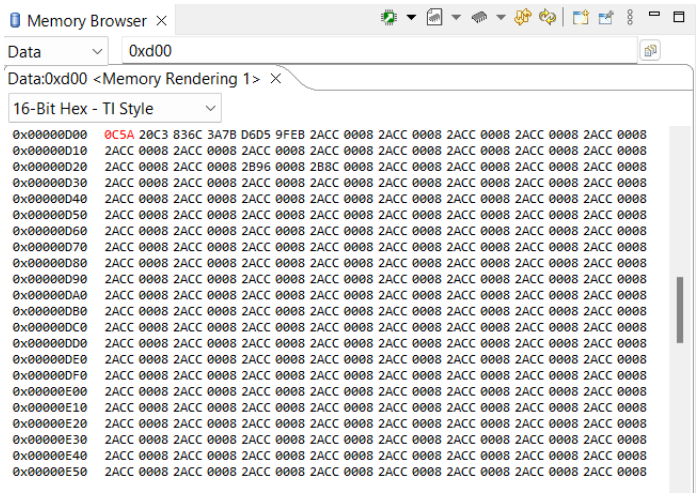

图 4-49. 配置 F2837xD controlCARD 以模拟 USB 引导

11. 复位 CPU，然后单击 ***Resume*** 启用 USB 引导序列。12. 如果 Boot ROM 中出现断点但没有可用的调试信息，或断点位于程序代码之外，请按照第 5.3 节加载 Boot ROM 符号。随后确认器件在 Boot ROM 中处于 USB 引导模式。

命令行选项和文件 I/O 可通过 C 标准库实现，但 USB 操作只能通过操作系统的器件驱动框架实现。目前有两个常用库可提供这一能力。

第一个是 libusb，这是一款采用 Unix 风格 API 的开源（LGPL）库。第二个是 WinUSB，它属于 Windows Driver Development Kit。两个库均在用户模式下运行，可通用访问 USB 器件，无需客户驱动程序。libusb 易于使用，也可在 Linux 上使用，但速度稍慢，且受许可证限制，分发较复杂。WinUSB 使用难度较高，但速度更快，生成的软件也更易于分发。

C2000Ware 附带的预编译 usb_flash_programmer.exe 使用 WinUSB，但也提供了两个库的源代码。USB 引导加载程序运行前，必须在器件上安装 WinUSB 驱动程序。

1. 打开设备管理器，在 *Universal Serial Bus controllers.* 下右键单击无法识别的器件。F28379D 显示为 *Stellaris Device Firmware Upgrade*。2. 依次选择 *Update Drivers > Browse my computer for drivers > Let me pick from a list of available drivers on my computer > Have Disk*。a. 在 *Copy from manufacturer's files from* 中输入以下内容：
C2000Ware_x_xx_xx_xx\utilities\flash_programmers\usb_flash_programmer\windows_driver\x86

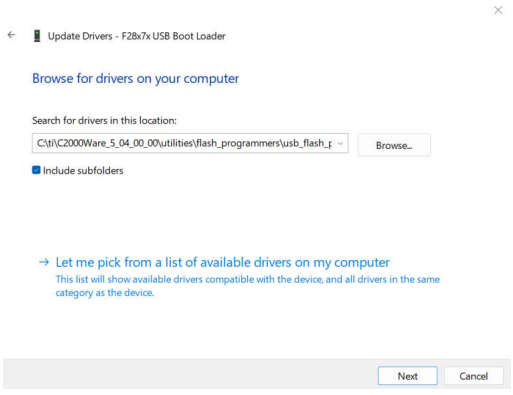

图 4-50. 浏览 C2000Ware 以查找 USB Windows 驱动程序

版权所有 © 2025 Texas Instruments Incorporated

3. 成功安装后，WinUSB 驱动程序会安装到器件上；器件显示为 *F28x7x USB Boot Loader*，位于 *Texas Instruments Microcontrollers* 类别下。

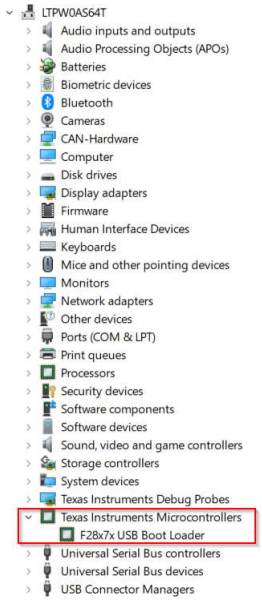

图 4-51. 在设备管理器中显示的 F28x7x USB 引导加载程序器件 4. 复位 CPU 并执行外部复位（XRSn），然后单击 ***Resume*** 开始引导序列。

命令行 PC 实用程序是一款轻量级编程工具（可执行文件约 64KB），便于集成到生产线编程等脚本环境中。该程序使用 Microsoft Visual Studio 和 C++ 编写。项目和源代码位于 C2000Ware (C2000Ware_x_xx_xx_xx > utilities > flash_programmers > usb_flash_programmer > src > VS2010_USBLoader2000)。

命令行输入文件路径按升序加载，因此必须先提供闪存内核文件路径，才能将内核加载到 RAM。闪存内核加载完成后，ROM 会移交控制权，内核开始执行并将应用程序写入闪存。闪存内核必须先完成器件的闪存编程准备，器件才能开始通信，因此需要短暂等待。在此期间，闪存内核会配置 PLL 和闪存等待状态。

下载开始时，系统会先传输一些预备数据，然后才传输实际的闪存应用程序代码，其中包括密钥、若干保留字段和应用程序入口点。内核收到入口点后才开始擦除闪存。闪存擦除可能需要几秒钟。请注意，即使应用程序加载过程看起来失败，闪存仍可能正在擦除。闪存擦除完成后，应用程序加载会继续，逐块传输应用程序代码并写入闪存。

1. 打开已编译的 usb_flash_programmer 可执行文件所在文件夹（C2000Ware_x_xx_xx_xx >utilities > flash_programmers > usb_flash_programmer）。
2. 打开终端，运行 usb_flash_programmer.exe，并使用以下命令：

```
usb_flash_programmer.exe <path_to_kernel_dat> <path_to_application_dat>
```

注意

闪存内核和闪存应用程序都必须采用前文第 4.1 节介绍的二进制引导格式。

应用程序写入闪存后，闪存内核会跳转到应用程序加载开始时传入的入口点，以尝试运行该应用程序。

版权所有 © 2025 Texas Instruments Incorporated

5 常见问题

本节针对用户尝试通过各种外设引导模式加载代码时遇到的常见障碍，提供相关建议。

5.1 通过软件选择 BMSP GPIO

问题：是否可以实现无需人工干预的软件引导加载流程?

回答：《C2000™ Software Controlled Firmware Update Process》应用报告 [7] 介绍了一种在 C2000 器件上使用现有引导模式的软件控制固件更新流程，无需人工选择引导模式。该文档介绍的方法可直接应用于 F28004x 器件，也可经过必要修改后用于旧版器件。

5.2 从闪存而非 RAM 运行闪存内核

问题：如何修改闪存内核，使其从闪存而非 RAM 运行?

回答：对于多闪存区器件，用户可在闪存中实现自定义引导加载程序，将应用程序代码直接加载到其他闪存区。对于单闪存区器件，可在闪存中实现自定义引导加载程序，以更新同一闪存区中的个别扇区，但 Flash API 必须在 RAM 中执行。

注意

不得在同一闪存区中执行 Flash API 函数及调用 Flash API 的应用程序函数。在同一闪存区执行 Flash API，可能会在编程和擦除数据时引入竞争条件。Flash API 必须在 RAM 或另一个闪存区（如果同一内核还有其他闪存区）中执行。[6]

使用自定义引导加载程序可让用户更灵活地控制引导流程，例如使用不同的外设 GPIO，而不受 Boot ROM 中预定义 GPIO 分配集合的限制。自定义引导加载程序还可减少引导加载步骤，因为无需先将中间闪存内核加载到 RAM 才能编程闪存。

此外，基于闪存的引导加载程序还可在固件升级失败时充当故障保护机制。引导到不会被升级的代码（即基于闪存的引导加载程序），可降低器件尝试引导损坏代码的可能性，因为引导加载程序可以通过多种方式验证现有固件（密钥值、校验和等）。基于 RAM 的闪存内核也能实现相同功能，但每次器件启动时都加载该内核并不实际。

实现自定义引导加载程序的一种方法，是修改 C2000Ware 中现有的闪存内核项目，使其从闪存而非 RAM 执行。这需要修改内核的链接器命令文件（该文件指定应用程序在存储器中的加载位置），并添加一个供编译器使用的预定义符号。

注意

以下步骤将修改 F28P55x SCI 闪存内核，使其从闪存执行；对于 C2000Ware 中已有闪存内核项目的任何 C2000 器件，也可采用相同流程。

版权所有 © 2025 Texas Instruments Incorporated

1. 右键单击项目并选择 *Add Files/Folders*。

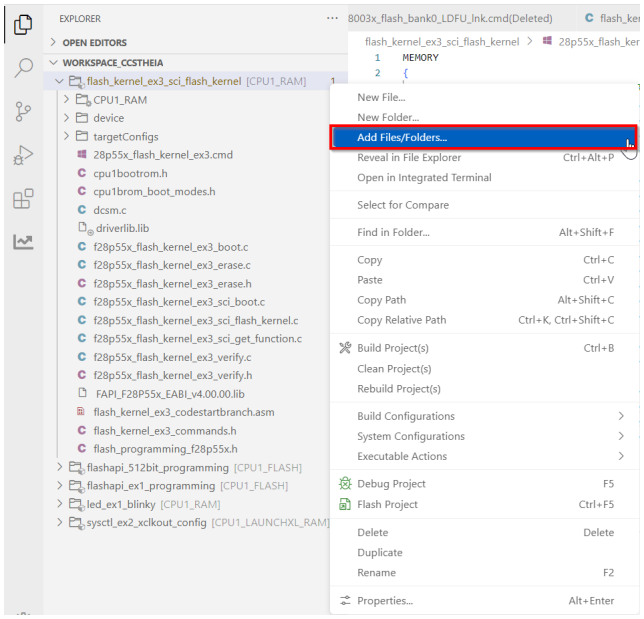

图 5-1. 在 CCS 项目中添加文件

2. 在 C2000Ware 中转到以下路径，并选择目标器件对应的通用闪存链接器命令文件。a. *C2000Ware_X_XX_XX_XX\device_support\DEVICE_NAME\common\cmd*

3. 右键单击 RAM 链接器命令文件并选择 *Exclude from Build*，以便在编译器构建过程中使用闪存链接器命令文件。

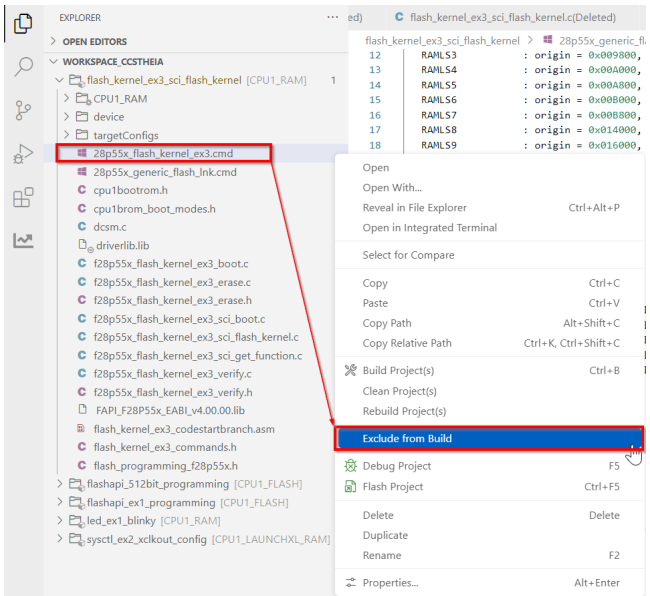

图 5-2. 从编译器构建中排除 RAM 链接器命令文件

版权所有 © 2025 Texas Instruments Incorporated

注意

使用通用闪存链接器命令文件时，内核会加载到闪存区 0 (entry point at 0x80000)。如果要将内核放到其他闪存区，请在链接器命令文件中将分配给闪存区 0 的 SECTIONS 修改为目标闪存区。

此外，还需要在链接器命令文件的 MEMORY 部分，将 BEGIN 更新为所需的闪存入口位置（codestart），并根据入口点处的 codestart 分配情况，调整闪存区的起始地址和长度设置。

有关链接器命令文件指令的详细信息，请参阅 [33]。

4. 右键单击项目，依次选择 *Build Configurations* > *Manage...*，创建新的构建配置。

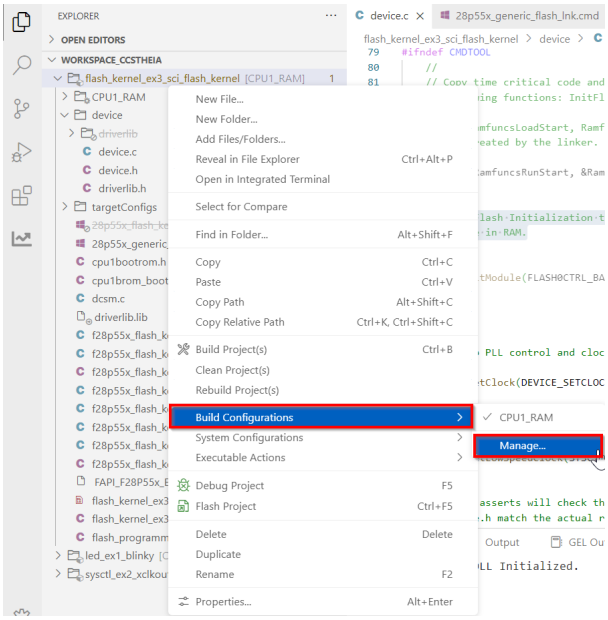

图 5-3. 为项目创建新的构建配置

a. 将构建配置命名为 *CPU1_FLASH*，并选择从 CPU1_RAM 配置复制设置。

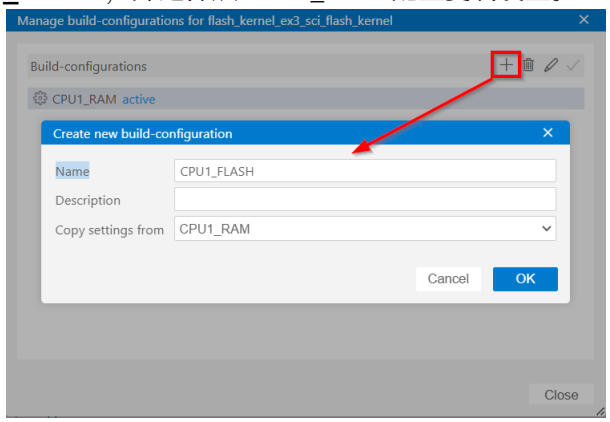

图 5-4. 创建 CPU1_FLASH 构建配置

5. 将活动构建配置设为 *CPU1_FLASH*。6. 在 device.c 中，时间关键代码和闪存设置代码（称为 *Ramfuncs*）会被复制到 RAM。此外，用于设置闪存等待状态的闪存初始化函数也必须位于 RAM 中。由于内核现在从闪存执行，因此必须定义符号 *_FLASH* 才能满足这些条件。a. 右键单击并导航到 *Properties* > *Tools* > *C2000 Compiler* > *Predefined Symbols*，然后定义符号 *_FLASH*。

版权所有 © 2025 Texas Instruments Incorporated

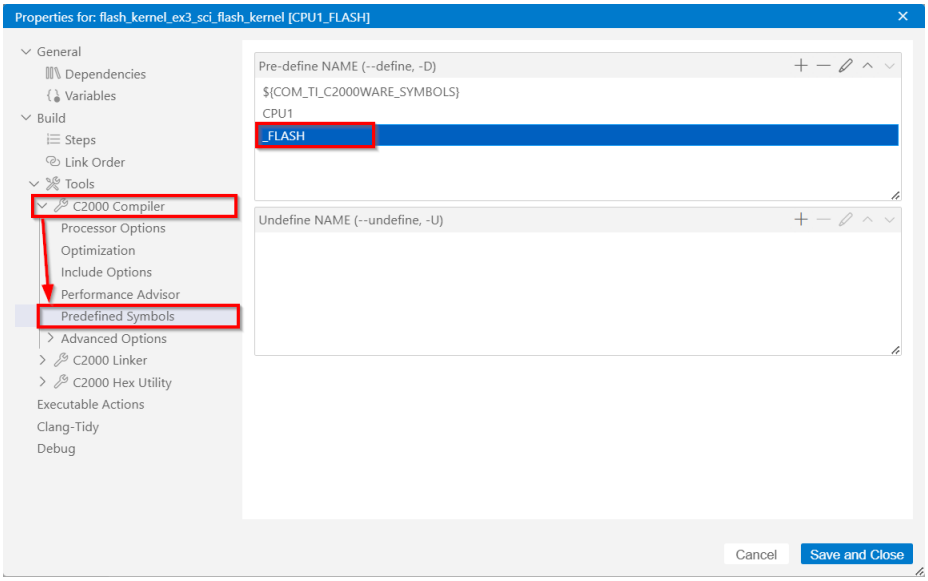

图 5-5. 添加预定义符号 _FLASH 以初始化闪存函数

7. 使用 *CPU1_FLASH* 构建配置重新构建闪存内核项目。这样，闪存内核会被构建为从闪存执行，并可编程到器件闪存中。

注意

由于内核已加载到器件上，因此必须使用 C2000Ware 中的 serial_flash_programmer_appIn.exe。该可执行文件的命令行参数与常规串行闪存编程器相同，但无需提供内核参数（-k）。

5.3 调试 Boot ROM 时未定义符号

问题：调试 Boot ROM 时，ROM 提示存在断点但没有可用的调试信息，或断点位于程序代码之外。这是什么原因?

回答：用户可将 Boot ROM 符号（.out 文件）加载到器件中，以便逐步执行器件 Boot ROM。加载符号是一种很有价值的调试方法。通过此选项，可将生成项目中的 *.out* 文件所含符号加载用于调试，而不是由 CCS 将实际的 *.out* 程序加载到内核中。因此，也可使用此方法调试 Boot ROM 或内置引导加载程序，获得更全面的运行信息。

1. 在 CCS 中打开一个工作区。

版权所有 © 2025 Texas Instruments Incorporated

2. 选择 View > Target Configurations。

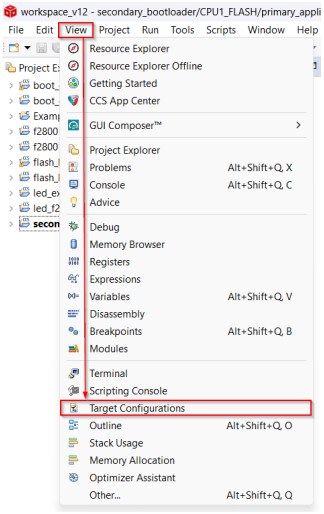

图 5-6. 在 C CS 中打开目标配置菜单

3. 可将该器件的项目导入 CCS，并使用该项目连接器件；也可从 C2000Ware 复制目标配置文件（.ccxml），路径为 (C2000Ware_x_xx_xx_xx > device_support > DEVICE_FAMILY > common > targetConfigs)，然后将其复制到 ***User Defined*** target configurations（本窗口中的 View > Target Configurations）。无论采用哪种方式，都需找到器件的目标配置文件，然后右键单击并手动启动：

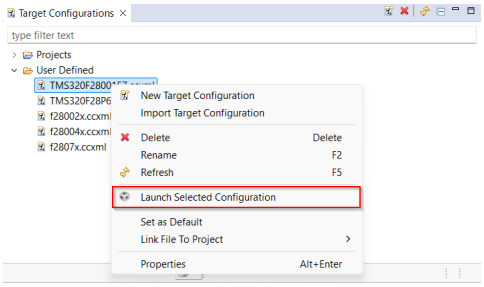

图 5-7. 在 CCS 中启动目标配置

4. CCS 打开调试窗口后，选择目标 CPU 并连接到目标器件。

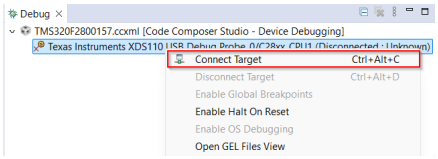

图 5-8. 连接到目标内核

5. 此时会弹出窗口，提示 Boot ROM 中存在断点，但没有可用的调试信息 ，或断点位于程序代码之外。

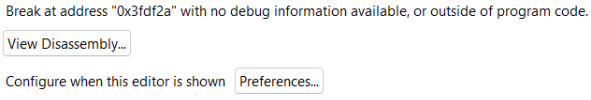

图 5-9. 未加载 Boot ROM 符号时的 CCS 视图 。

版权所有 © 2025 Texas Instruments Incorporated

6. 转到工具栏并单击 *Load Symbols*。

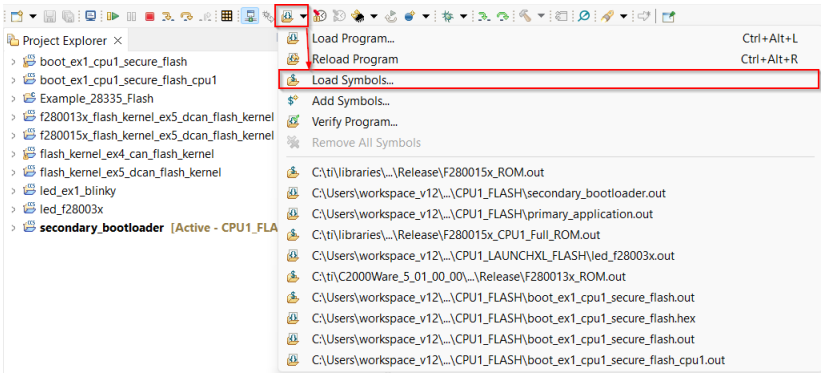

图 5-10. 在 CCS 中找到“加载符号”选项

7. 加载 Boot ROM *.out* 文件。该文件位于 C200Ware 中：a. *C2000Ware_x_xx_xx_xx > libraries > libraries > boot_rom > DEVICE_FAMILY > REV# > rom_sources > CPU# > ccs_files > Release*

8. 如果弹出窗口提示找不到源文件，请选择 *Locate File*，并在 C2000Ware 中查找：

a. *C2000Ware_x_xx_xx_xx > libraries > boot_rom > DEVICE_FAMILY > REV# > rom_sources > CPU# > DEVICE_FAMILY_ROM > bootROM > source*

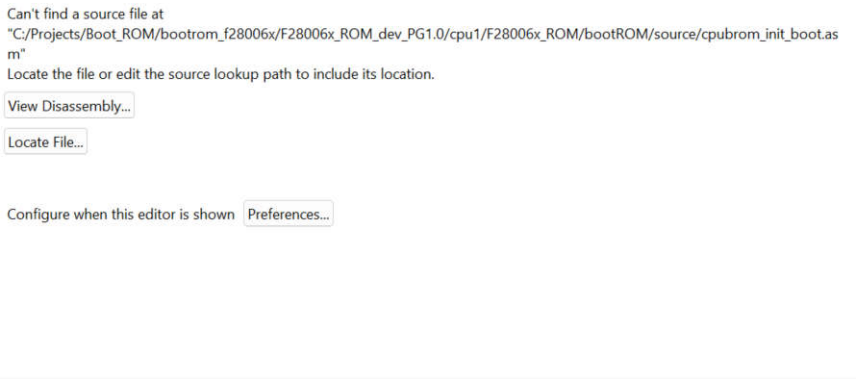

图 5-11. 查找 Boot ROM 源文件

打开该文件后，可查看 Boot ROM 当前指令，并逐步调试、复位和重新启动器件。

5.4 在 OTP 中写入数值并使用片上闪存 工具

本节通过两个示例用例，介绍如何使用片上闪存工具对 OTP 编程。

Note

Although this section is based on F280015x devices, the same flow can be applied to any C2000 device that supports custom BMSPs and boot mode tables.

Device specific information can be found in the *Boot ROM* chapter of the device's Technical Reference Manual (TRM).

1. 启动目标 CPU 内核的调试会话后，打开片上闪存工具（参见第 3 节，了解 如何在 CCS 中找到该工具）。2. 在 Zone 1 或 Zone 2 下找到 GPREG(BOOTCTRL)（请注意，Zone 2 的优先级高于 Zone 1）。随后可向 OTP-BOOTPIN-CONFIG 和 OTP-BOOTDEF-LOW/OTP-BOOTDEF-HIGH 寄存器写入数据。

版权所有 © 2025 Texas Instruments Incorporated

a. 表 2-7 列出了 F280015x 器件的引导配置寄存器位置，可在器件专用 TRM 的 *Boot ROM Configuration Registers* 部分中查到。*Register Name* 列显示片上闪存工具所引用的 Boot ROM 寄存器名称。

示例 1：未选择引导模式的引脚

此用例展示了一个应用场景：应用程序不希望使用任何 BMSP，并始终引导到闪存入口地址 0x88000。

有关表中应设置数值的说明，请参阅器件 TRM 的 *GPIO Assignments* 部分。对于闪存入口点，请参阅 TRM 中介绍不同引导模式入口地址的 *Entry Points* 部分。这些入口地址指示 Boot ROM 在引导结束时，根据所选引导模式跳转到相应位置。

1. 按如下方式编程 OTP 中的 BOOTPIN_CONFIG 位置：
   a. 将 BOOTPIN_CONFIG.BMSP0 设为 0xFF（禁用）。b. 将 BOOTPIN_CONFIG.BMSP1 设为 0xFF（禁用）。c. 将 BOOTPIN_CONFIG.BMSP2 设为 0xFF（禁用）。d. 将 BOOTPIN_CONFIG.KEY 设为 0x5A，使 Boot ROM 将这些寄存器位视为有效，并使用自定义引导表。

2. 为器件配置 BOOTDEF 位置中的选项。这实际上会设置器件专用的引导模式表。
   a. 将 BOOTDEF.BOOTDEF0 设为 0x23，以引导到闪存（入口地址选项 1）。这样，闪存引导会使用引导表索引 0。

图 5-12 展示了片上闪存工具中已填写完成的输入字段，用于编程此示例。

图 5-12. 示例 1：已编程的闪存插件引导配置

表 5-1. 零引脚引导配置结果

BMSP Index	BOOTDEF
0	0x23 (Flash Boot to address 0x88000)

示例 2：使用两个引导模式选择引脚

此用例展示了更常见的应用场景：使用两个引导模式选择引脚，在自定义引导表中选择 CAN、安全闪存或 SCI 引导。

有关表中应设置数值的说明，请参阅器件 TRM 的 *GPIO Assignments* 部分。对于闪存入口点，请参阅 TRM 中介绍不同引导模式入口地址的 *Entry Points* 部分。这些入口地址指示 Boot ROM 在引导结束时，根据所选引导模式跳转到相应位置。

1. 按如下方式编程 OTP 中的 BOOTPIN_CONFIG 位置：
   a. 将 BOOTPIN_CONFIG.BMSP0 设为用户指定的 GPIO，例如将 0x2A 用于 GPIO42。b. 将 BOOTPIN_CONFIG.BMSP1 设为用户指定的 GPIO，例如将 0x58 用于 GPIO88。c. 将 BOOTPIN_CONFIG.BMSP2 设为 0xFF（禁用）。d. 将 BOOTPIN_CONFIG.KEY 设为 0x5A，使 Boot ROM 将这些寄存器位视为有效，并使用自定义引导表。

版权所有 © 2025 Texas Instruments Incorporated

2. 为器件配置 BOOTDEF 位置中的选项。这实际上会设置器件专用的引导模式表。

a. 将 BOOTDEF.BOOTDEF0 设为 0x6A，以启用安全闪存引导（入口地址选项 3）。这样，安全闪存引导会使用引导表索引 0。b. 将 BOOTDEF.BOOTDEF1 设为 0x22，以使用 CAN 引导选项 1。这样，CAN 引导会使用引导表索引 1。c. 将 BOOTDEF.BOOTDEF2 设为 0x41，以使用 SCI 引导选项 2。这样，CAN 引导会使用引导表索引 2。

图 5-13 展示了片上闪存工具中已填写完成的输入字段，用于编程此示例。

图 5-13. 示例 2：已编程的闪存插件引导配置

表 5-2. 双引导模式选择引脚配置

BMSP 索引	**BOOTDEF**
0	0x6A（安全闪存引导至地址 0x90000）
1	0x22（CAN 引导 1，使用备用 GPIO）
2	0x41（SCI 引导 2，使用备用 GPIO）
3	无需关心（未使用）

5.5 使用闪存 API 插件在 OTP 中写入数值

注意

虽然本节以 F280015x 器件为例，但支持自定义 BMSP 和引导模式表的任何 C2000 器件都可采用相同流程。

器件专用信息可在该器件技术参考手册（TRM）的 *Boot ROM* 章中查找。

第 5.5 节介绍的自定义引导配置也可使用编译器的 RETAIN 和 DATA_SECTION 编译指示写入 DCSM OTP [31]。以下步骤采用示例 2（使用两个引导模式选择引脚）的配置。

1. 在 CCS 中打开任意 C2000Ware 示例，并在 main 函数上方添加以下代码片段（可放在文件中任何函数之外的位置）。这些代码行分别用于编程 BOOTPIN-CONFIG 和 BOOTDEF-LOW 寄存器。

```
#pragma RETAIN(otp_z1_data_1)  #pragma DATA_SECTION(otp_z1_data_1,"
dcsm_zsel_z1");  const long otp_z1_data_1 = 0x5AFF582A;  #pragma RETAIN(
otp_z1_data_2) #pragma DATA_SECTION(otp_z1_data_2,"dcsm_zsel_z1_2");
const long otp_z1_data_2 = 0xFF41226A;
```


版权所有 © 2025 Texas Instruments Incorporated

2. 在项目的链接器命令文件中添加以下行，以定义 DCSM OTP BOOTPIN-CONFIG 和 BOOTDEF-LOW 的存储器映射位置；这些位置与器件专用 TRM 的 *Boot ROM Configuration Registers* 节中定义的一致。以下地址适用于 F280015x 器件上的 DCSM 用户 OTP 引导配置寄存器。

```
MEMORY {     PAGE 0:       DCSM_ZSEL_Z1_P0: origin = 0x078008, length = 0x000002 // Z1-OTP-BOOTPIN-CONFIG
DCSM_ZSEL_Z1_P1: origin = 0x07800C, length = 0x000002 // Z1-OTP-BOOTDEF-LOW }   SECTIONS {     dcsm_zsel_z
1_1 : > DCSM_ZSEL_Z1_P0, PAGE = 0     dcsm_zsel_z1_2 : > DCSM_ZSEL_Z1_P1, PAGE = 0  }
```

3. 重新构建示例，并通过 CCS 中的 JTAG 将其加载到目标器件。CCS 的程序加载器和 Flash API 插件会负责将这些值写入 OTP 位置。必须谨慎选择并写入这些值，因为 OTP 位置无法重新编程。

6 总结

在嵌入式处理器的应用中，尤其是在现场部署的系统里，如果无法可靠地使用 JTAG 调试探针，通常就需要灵活的编程方式。C2000 微控制器通过在 Boot ROM 中提供引导加载工具，支持设备固件升级。这些引导加载程序可通过多种通信接口，将应用代码从外部主机加载到 RAM；随后还可使用闪存内核对片上闪存进行编程。本文介绍基本的引导加载配置，并详细说明如何利用常见引导模式将应用代码加载到 C2000 器件中。

版权所有 © 2025 Texas Instruments Incorporated

7 参考文献

1. Texas Instruments, 《Code Composer Studio™ integrated development environment (IDE)》, 网页；2. Texas Instruments, 《UniFlash flash programming tool》, 网页；3. Texas Instruments, 《C2000Ware for C2000 MCUs》, 网页；4. Texas Instruments, [E2E]《FAQ on Flash software for C2000 MCUs》, 网页；5. Texas Instruments, [E2E]《FAQ on C2000 CCS Flash plugin and UniFlash》, 网页；6. Texas Instruments, [E2E]《FAQ on Flash API usage for C2000 devices》, 网页；7. Texas Instruments, 《C2000™ Software Controlled Firmware Update Process》, 应用报告；8. Texas Instruments, 《C28x Academy, Dual Code Security Module (DCSM)》, 网页；9. Texas Instruments, 《C2000™ DCSM Security Tool》, 应用报告；10. Texas Instruments, 《C2000 Multicore Development User Guide》；11. Texas Instruments, 《TMS320C28x Assembly Language Tools》, 用户指南；12. Texas Instruments, 《TI Arm Clang Compiler Tools User's Guide - Linker Description》；13. Texas Instruments, 《Secure BOOT on C2000 Device》, 应用报告；14. Texas Instruments, 《Serial Flash Programming of C2000™ Microcontrollers》, 应用说明；15. Texas Instruments, 《Introduction to the Controller Area Network (CAN)》, 应用报告；16. Texas Instruments, 《CAN Flash Programming of C2000 Microcontrollers》, 应用说明；17. PEAK System, 《PCAN-Basic API》；18. Texas Instruments, 《Getting Started with the MCAN (CAN FD) Module》, 应用说明；19. Texas Instruments, [E2E]《TMS320F28377D: 28377D USB Bootloader》, 网页；20. Texas Instruments, 《USB Flash Programming of C2000™ Microcontrollers》, 应用报告；21. Texas Instruments, 《XDS100 Debug Probe》, 产品页面；22. Texas Instruments, 《Delfino™ TMS320F28379D controlCARD R1.3》, 用户指南；23. Texas Instruments, 《TMS320F28003xD Real-Time Microcontrollers》, 技术参考手册；24. Texas Instruments, 《TMS320F280015x Real-Time Microcontrollers》, 技术参考手册；25. Texas Instruments, 《TMS320F28P65x Real-Time Microcontrollers》, 技术参考手册；26. Texas Instruments, 《TMS320F2837xD Real-Time Microcontrollers》, 技术参考手册；27. Texas Instruments, 《TMS320F28P55x Real-Time Microcontrollers》, 技术参考手册；28. Texas Instruments, 《TMS320F2838xD Real-Time Microcontrollers With Connectivity Manager Technical Reference Manual (SPRUII0)》；29. Texas Instruments, 《TMS320F280015x Real-Time Microcontrollers》, 数据表；30. Texas Instruments, 《TMS320F280039x Real-Time Microcontrollers》, 数据表；31. Texas Instruments, 《F021 Flash API Version 2.01.01》, 参考指南；32. Texas Instruments, 《TMS320F280013x/15x Flash API Version 2.00.10.00》, 参考指南；33. Texas Instruments, 《TI Linker Command File Primer》, 网页；34. Texas Instruments, 《TMS320C28x Optimizing C/C++ Compiler》, 用户指南；35. Texas Instruments, [E2E]《LAUNCHXL-F2800157 CAN Flash Programming》, 网页。

版权所有 © 2025 Texas Instruments Incorporated

重要声明与免责声明

TI 按“原样”提供技术和可靠性数据（包括数据表）、设计资源（包括参考设计）、应用或其他设计建议、Web 工具、安全信息及其他资源，并且不承担由此产生的任何责任。TI 不作任何明示或暗示的保证，包括但不限于对适销性、特定用途适用性或不侵犯第三方知识产权的任何暗示保证。

这些资料面向使用 TI 产品进行设计的专业开发人员。您须自行负责：(1) 为应用选择合适的 TI 产品，
(2) 设计、验证和测试应用，以及 (3) 确保应用符合适用标准和其他安全、信息安全、法规或其他要求。

这些资料如有变更，恕不另行通知。TI 仅许可您将这些资料用于开发采用资料中所述 TI 产品的应用。禁止以其他方式复制或展示这些资料。TI 不授予任何其他 TI 知识产权或第三方知识产权的许可。对于因您使用这些资料而产生的任何索赔、损害、成本、损失和责任，TI 概不负责；您须对 TI 及其代表作出全额赔偿。

TI 产品受 TI 销售条款或其他适用条款约束；相关条款可在 ti.com 上查阅，或随 TI 产品一并提供。TI 提供这些资料不会扩大或以其他方式更改 TI 对产品适用的保证或免责声明。

TI 反对并拒绝您可能提出的任何附加条款或不同条款。 重要声明

Mailing Address: Texas Instruments, Post Office Box 655303, Dallas, Texas 75265
Copyright © 2025, Texas Instruments Incorporated